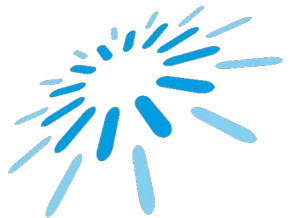




Collaboration, Quality, Test

Prof. Dr. Jóakim von Kistowski



TH Aschaffenburg
university of applied sciences

Recap:
Why is testing a **sample**?

Remember:

There are no (comprehensive) systems that are error-free! Nor can any test prove that they are error-free!

Learning from Errors

Errors are a source for learning experiences

- How could the error have been **prevented**
(→ constructive, organizational QA measures)
- How could you have **found** the error (→ analytic QA measures)

Debugging vs. Test

What is the difference?
Use the terms **defect** and **failure**!

Debugging vs. Test

- **Debugging**, Goal: localizing the **defect**, requires developer skills.
- **Testing**, Goal: find **Failures**

Verification vs. Validation

What is the difference?

Validation vs. Verification (4/6)

Which is which? – **Requirements:**



Requirements met?

Verification ✓

Made of plastic,
blue

Vessel to be
filled with water

Vessel to be
filled with water

With handle

Validation vs. Verification (5/6)

Which is which? – **Requirements:**



Usable?

Validation **X**

Made of plastic,
blue

Vessel to be
filled with water

Vessel to be
filled with water

With handle

Test and Quality

How do they relate to one another?

Testing and Quality

Testing measures Quality

- e.g. based on the number of errors found.

Testing indirectly increases quality

- as errors (conditions) can be detected and corrected before delivery.

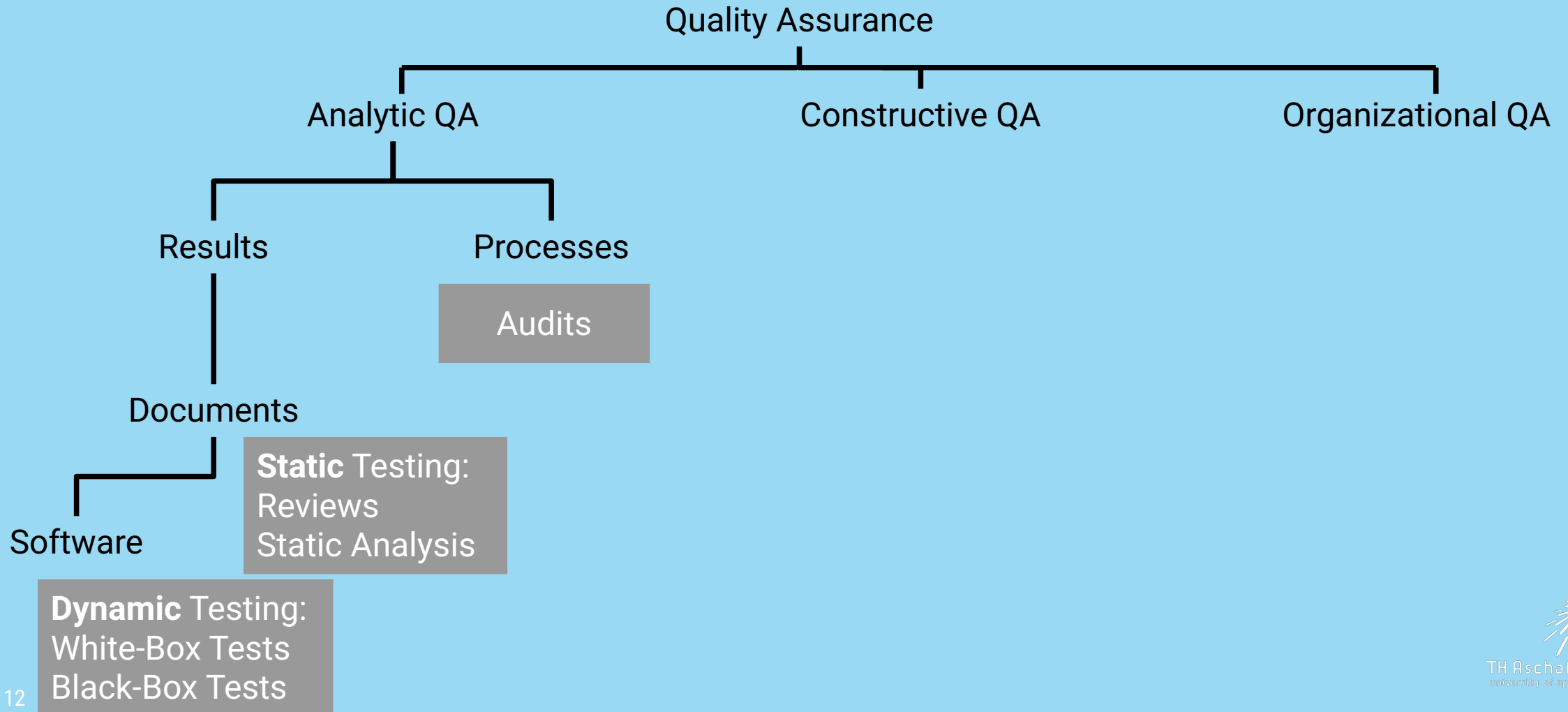
Testing indirectly increases process quality

- as errors can be documented, analyzed and thus errors can be avoided in the future.

Testing increases confidence in the quality of the system.

- when few or no errors are found.

Quality Assurance – QA



Remember:

Ensuring quality costs money. Not delivering quality may cost even more money. **Prioritize** the quality attributes and corresponding QA measures carefully!

Chapter 1: Fundamentals of Testing



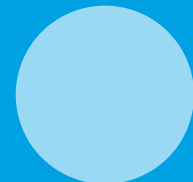
Terms and Motivation

Testing Principles

Test Activities, Testware and Test Roles

Test Cases, Expected Results, and Test Oracle

Essential Skills



Learning Objectives for Fundamentals of Testing

(according to Certified Tester Foundation Level Syllabus, English Version V4.0)

- **1.1 What is Testing?**
- FL-1.1.1 (K1) Identify typical test objectives
- FL-1.1.2 (K2) Differentiate testing from debugging
- **1.2 Why is Testing Necessary?**
- FL-1.2.1 (K2) Exemplify why testing is necessary
- FL-1.2.2 (K1) Recall the relation between testing and quality assurance
- FL-1.2.3 (K2) Distinguish between root cause, error, defect, and failure

Learning Objectives for Fundamentals of Testing

(according to Certified Tester Foundation Level Syllabus, English Version V4.0)

- **1.3 Testing Principles**

- FL-1.3.1 (K2) Explain the seven testing principles

- **1.4 Test Activities, Testware and Test Roles**

- FL-1.4.1 (K2) Summarize the different test activities and tasks
- FL-1.4.2 (K2) Explain the impact of context on the test process
- FL-1.4.3 (K2) Differentiate the testware that supports the test activities
- FL-1.4.4 (K2) Explain the value of maintaining traceability
- FL-1.4.5 (K2) Compare the different roles in testing

- **1.5 Essential Skills and Good Practices in Testing**

- FL-1.5.1 (K2) Give examples of the generic skills required for testing
- FL-1.5.2 (K1) Recall the advantages of the whole team approach
- FL-1.5.3 (K2) Distinguish the benefits and drawbacks of independence of testing

Testing Principles (Overview)

The following principles have established themselves in the last 50 years. They can be used as guidelines:

- 1) Testing shows the presence, not the absence of defects.
- 2) Exhaustive testing is impossible. Testing everything is not feasible except in trivial cases.
- 3) Early testing saves time and money.
- 4) Defects cluster together.
- 5) Tests wear out.
- 6) Testing is context dependent.
- 7) Absence-of-defects fallacy.

Testing Principles (1/6)

- Principle 1: **Testing shows the presence, not the absence of defects.**

Testing can show that defects are present in the test object, but cannot prove that there are no defects. Testing reduces the probability of defects remaining undiscovered in the test object, but even if no defects are found, testing cannot prove test object correctness.

**“Program testing can be used
to show the presence of
bugs, but never to show their
absence!”**

Dijkstra, 1970

Testing Principles (2/6)

- Principle 2: **Exhaustive testing is impossible. Testing everything is not feasible except in trivial cases.**

Rather than attempting to test exhaustively, test techniques, test case prioritization, and risk-based testing, should be used to focus test efforts.

Testing Principles (2/6)

- Principle 2: **Exhaustive testing is impossible. Testing everything is not feasible except in trivial cases.**

Rather than attempting to test exhaustively, test techniques, test case prioritization, and risk-based testing, should be used to focus test efforts.

Testing Principles (3/6)

- Principle 3: **Early testing saves time and money.**

Defects that are removed early in the process will not cause subsequent defects in derived work products. The cost of quality will be reduced since fewer failures will occur later in the SDLC. To find defects early, both static testing and dynamic testing should be started as early as possible.

Testing Principles (4/6)

- Principle 4: **Defects cluster together.**

A small number of system components usually contain most of the defects discovered or are responsible for most of the operational failures. This phenomenon is an illustration of the Pareto principle. Predicted defect clusters, and actual defect clusters observed during testing or in operation, are an important input for risk-based testing.

Testing Principles (5/6)

- Principle 5: **Tests wear out.**

If the same tests are repeated many times, they become increasingly ineffective in detecting new defects. To overcome this effect, existing tests and test data may need to be modified, and new tests may need to be written. However, in some cases, repeating the same tests can have a beneficial outcome, e.g., in automated regression testing.

- Principle 6: **Testing is context dependent.**

There is no single universally applicable approach to testing. Testing is done differently in different contexts.

Testing Principles (6/6)

- Principle 7: **Absence-of-defects fallacy.**

It is a fallacy (i.e., a misconception) to expect that software verification will ensure the success of a system. Thoroughly testing all the specified requirements and fixing all the defects found could still produce a system that does not fulfill the users' needs and expectations, that does not help in achieving the customer's business goals, and that is inferior compared to other competing systems. In addition to verification, validation should also be carried out.

Testing Principles (Overview)

The following principles have established themselves in the last 50 years. They can be used as guidelines:

- 1) Testing shows the presence, not the absence of defects.
- 2) Exhaustive testing is impossible. Testing everything is not feasible except in trivial cases.
- 3) Early testing saves time and money.
- 4) Defects cluster together.
- 5) Tests wear out.
- 6) Testing is context dependent.
- 7) Absence-of-defects fallacy.

When do you stop testing?

Testing Effort

“Testing makes economic sense as long as the costs of finding and eliminating an error in the test are lower than the costs associated with the occurrence of an error during use.”

after: M. Pol, T. Koomen, A. Spillner: Management und Optimierung des Testprozesses. Dpunkt-Verlag, 2. Auflage, 2002



“A good test is like liability insurance: it costs real money, but lets the project manager and the customer sleep soundly. A good night's sleep also includes good insurance that covers all possible risks. Confidence in the software requires a good test that covers the entire reality of production.”

after: Siedersleben, J. (Hrsg): Softwaretechnik, Praxiswissen für Softwareingenieure. 2. Auflage, Hanser, 2003

Remember:

The testing effort must be reasonably proportional to the achievable result.

Testing Effort

Testing is subject to limited resources (especially time, if testing is only tackled at the end of development)

Particularly important:

- Select **adequate** test procedures
(suitable for the test object and the quality objectives)
→ We'll take a look in the lecture
- Avoid **unnecessary tests** (which do not provide any new findings)
→ Review test cases regularly
- Consider both positive and **negative test cases**
- Tests for **not explicitly required** functionality (possibly even intentionally harmful)
must also be included

Prioritizing Tests



Tests cost money



Exhaustive testing
is impossible!
Requires **YEARS**,
even for small
examples!



Requirements
regarding quality
increase
(both functional and
non-functional)

Question

What criteria should you use when prioritizing?

Criteria for Prioritizing (1/3)

- Expected error severity or amount of **damage** (analytics vs. core banking)
- **Probability of occurrence** of a failure (new team, new technology, etc.)
- Combination of amount of damage and probability of occurrence (**risk** = amount of damage * probability of occurrence)
- **Perception** of failure (color wrong / not CI-compliant → “Not a key feature” but in the customer's perception it can be a show stopper)

Criteria for Prioritizing (2/3)

- **Priority of requirements** by the customer
(the more important a requirement, the more important the tests)
- Weighting of the **quality attributes** by the customer
(For high-priority quality requirements, plan intensive tests, e.g. performance)
- Priority of test cases from a development perspective
(large damage impact radius and/or complex first)
- High project risk
- A bug rarely arrives alone! Bugs are social creatures! Retain the ability to change priorities!

Criteria for Prioritizing (3/3)



- Bugs are social beings ;-)
- If you find an error, there are likely more!
- Ability to change priority is important!

**Prioritize tests so that
whenever you stop testing
you have done the best testing
in the time available.**

Source (2013): <http://kehittaminen.turkuamk.fi/msuni/Testing/TestingCourse03-01-slides.pdf>

Chapter 1: Fundamentals of Testing



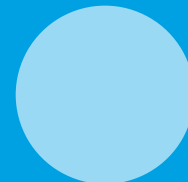
Terms and Motivation

Testing Principles

Test Activities, Testware and Test Roles

Test Cases, Expected Results, and Test Oracle

Essential Skills



Learning Objectives for Fundamentals of Testing

(according to Certified Tester Foundation Level Syllabus, English Version V4.0)

- **1.3 Testing Principles**
- FL-1.3.1 (K2) Explain the seven testing principles
- **1.4 Test Activities, Testware and Test Roles**
- FL-1.4.1 (K2) Summarize the different test activities and tasks
- FL-1.4.2 (K2) Explain the impact of context on the test process
- FL-1.4.3 (K2) Differentiate the testware that supports the test activities
- FL-1.4.4 (K2) Explain the value of maintaining traceability
- FL-1.4.5 (K2) Compare the different roles in testing
- **1.5 Essential Skills and Good Practices in Testing**
- FL-1.5.1 (K2) Give examples of the generic skills required for testing
- FL-1.5.2 (K1) Recall the advantages of the whole team approach
- FL-1.5.3 (K2) Distinguish the benefits and drawbacks of independence of testing

Task



Find the definitions for the following terms in the ISTQB glossary. What do they mean?

test basis

test
execution
schedule

test
condition

test case

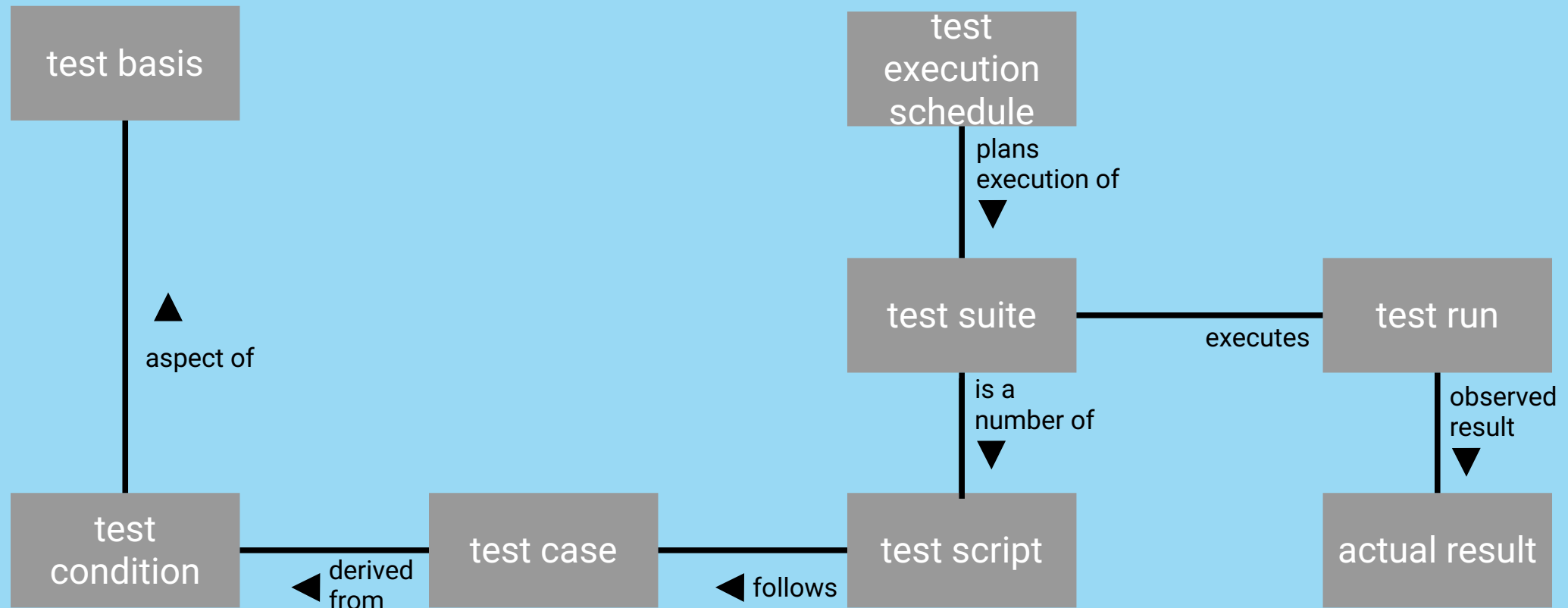
test run

test suite

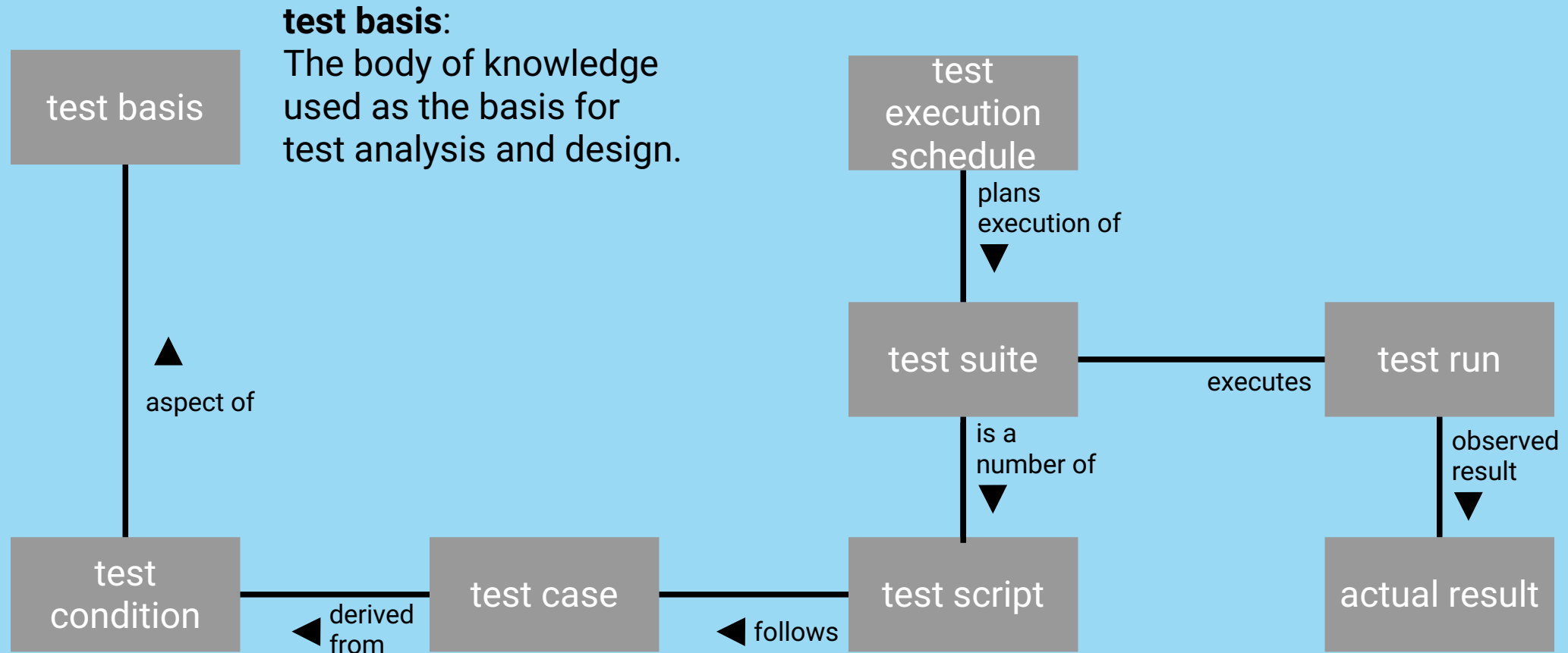
test script

actual result

Test Process Artifacts (1/4)

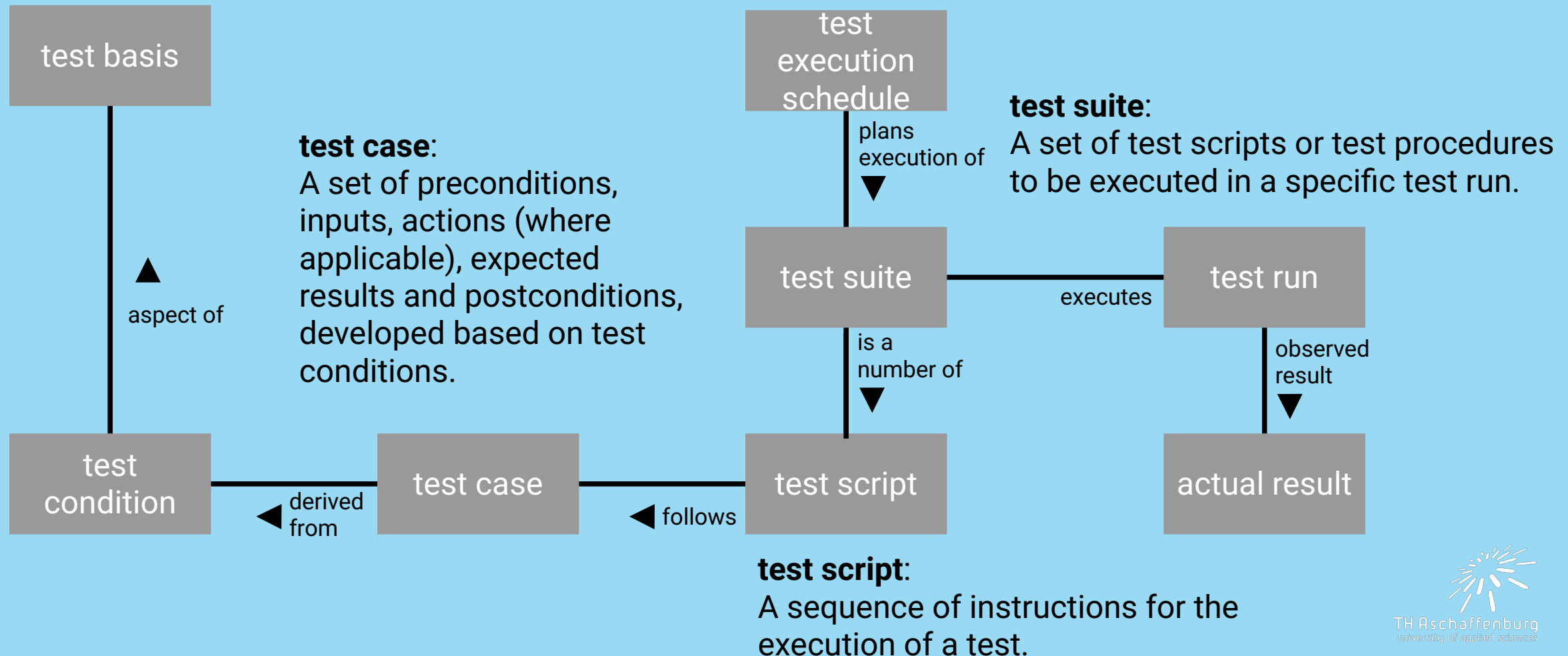


Test Process Artifacts (2/4)

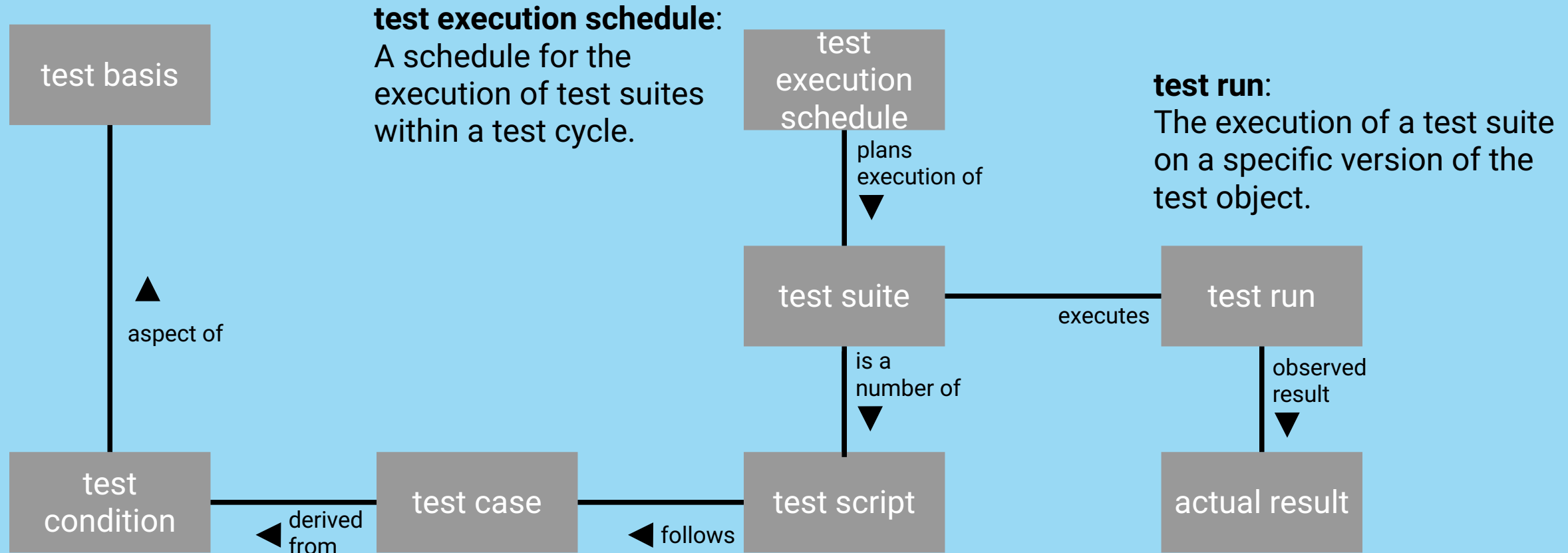


test condition:
A testable aspect of a component or system identified as a basis for testing.

Test Process Artifacts (3/4)



Test Process Artifacts (4/4)

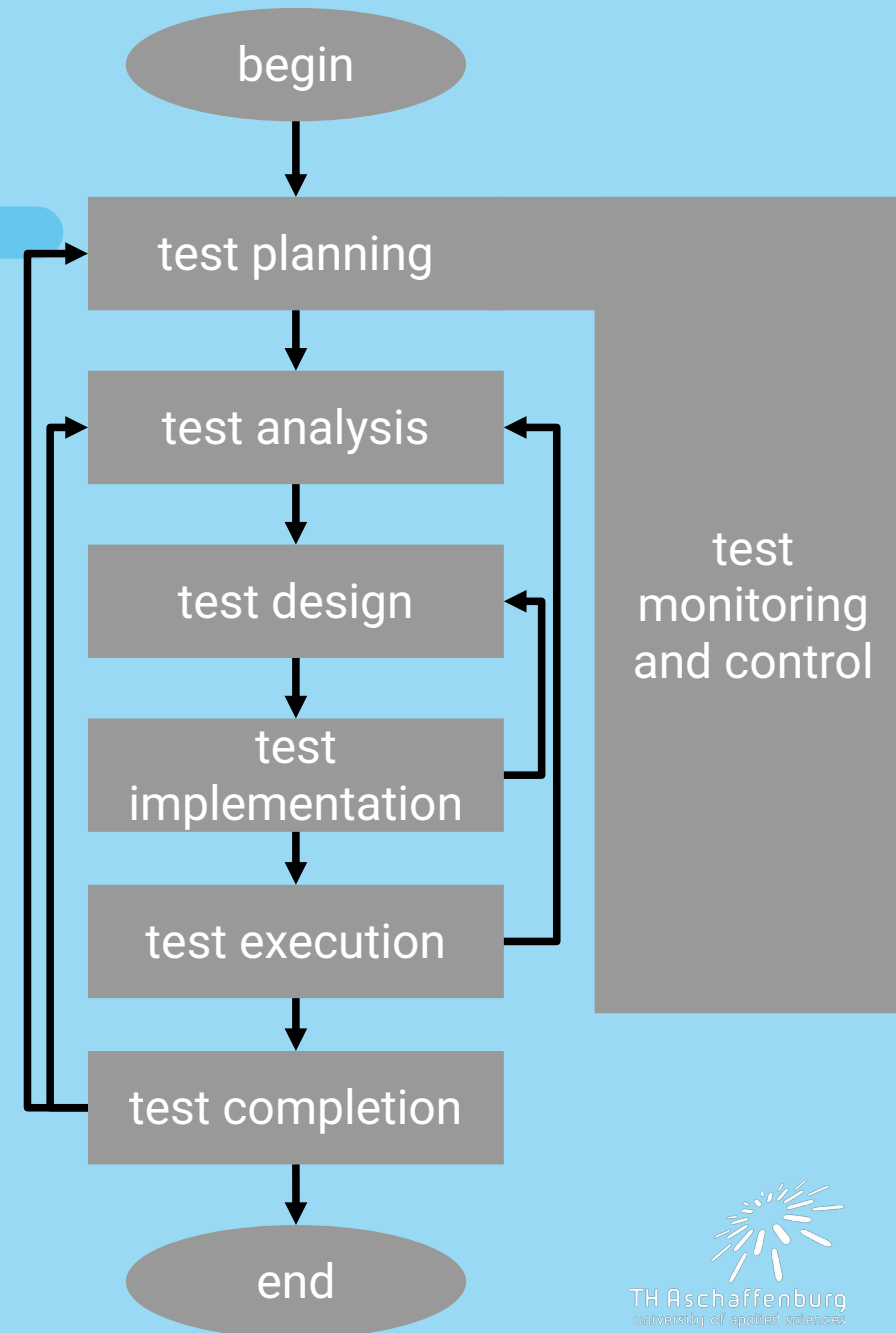


Test Activities according to CTFL

- Test planning
- Test monitoring and control
- Test analysis
- Test design
- Test implementation
- Test execution
- Test completion

Some of these activities are carried out overlapping or in parallel.

The test process must be designed appropriately for each test stage.



7 Teams (one team for each test activity):

What are the main goals/objectives of the given test activity?

What are the concrete (sub-)activities?

Which of the test process artifacts correspond to the given test activity?

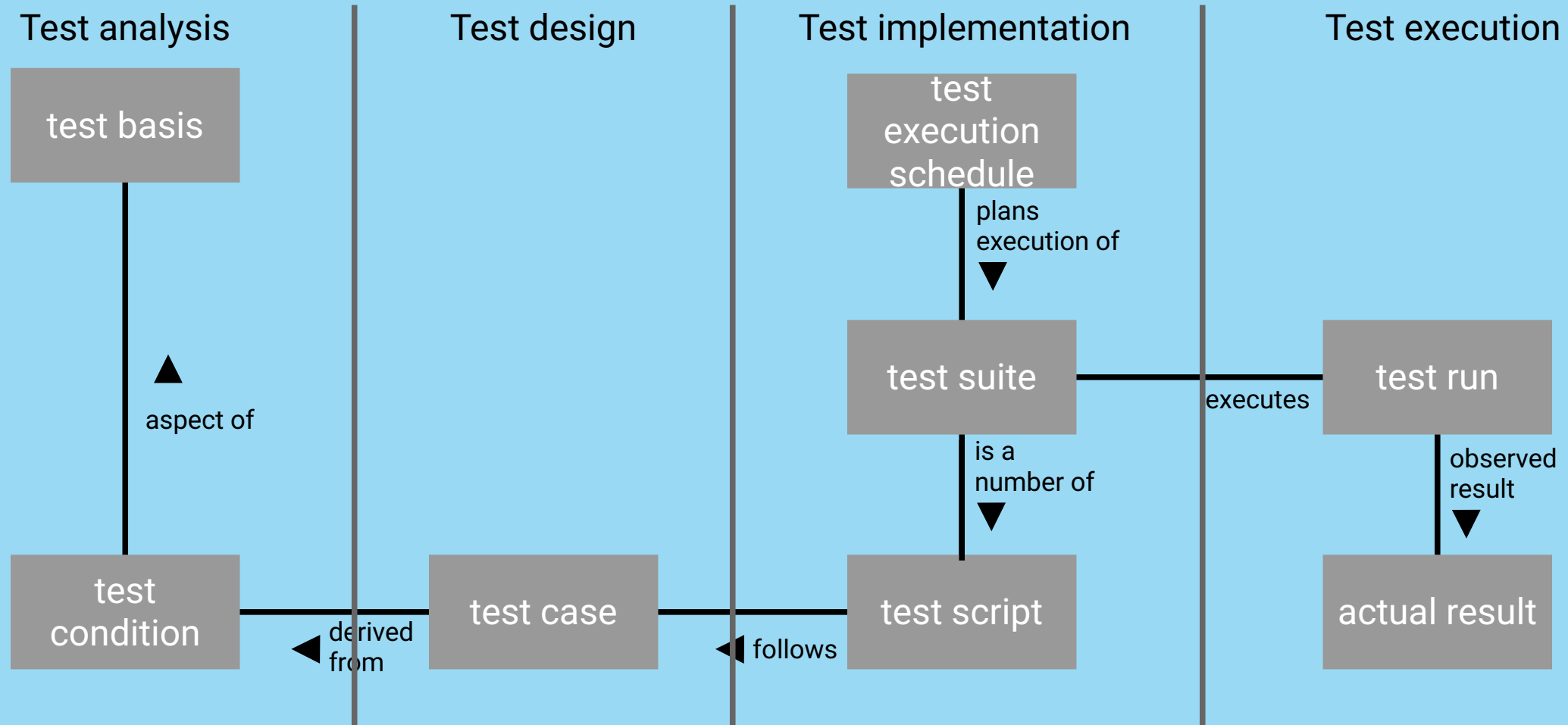


ITSQB
Glossary



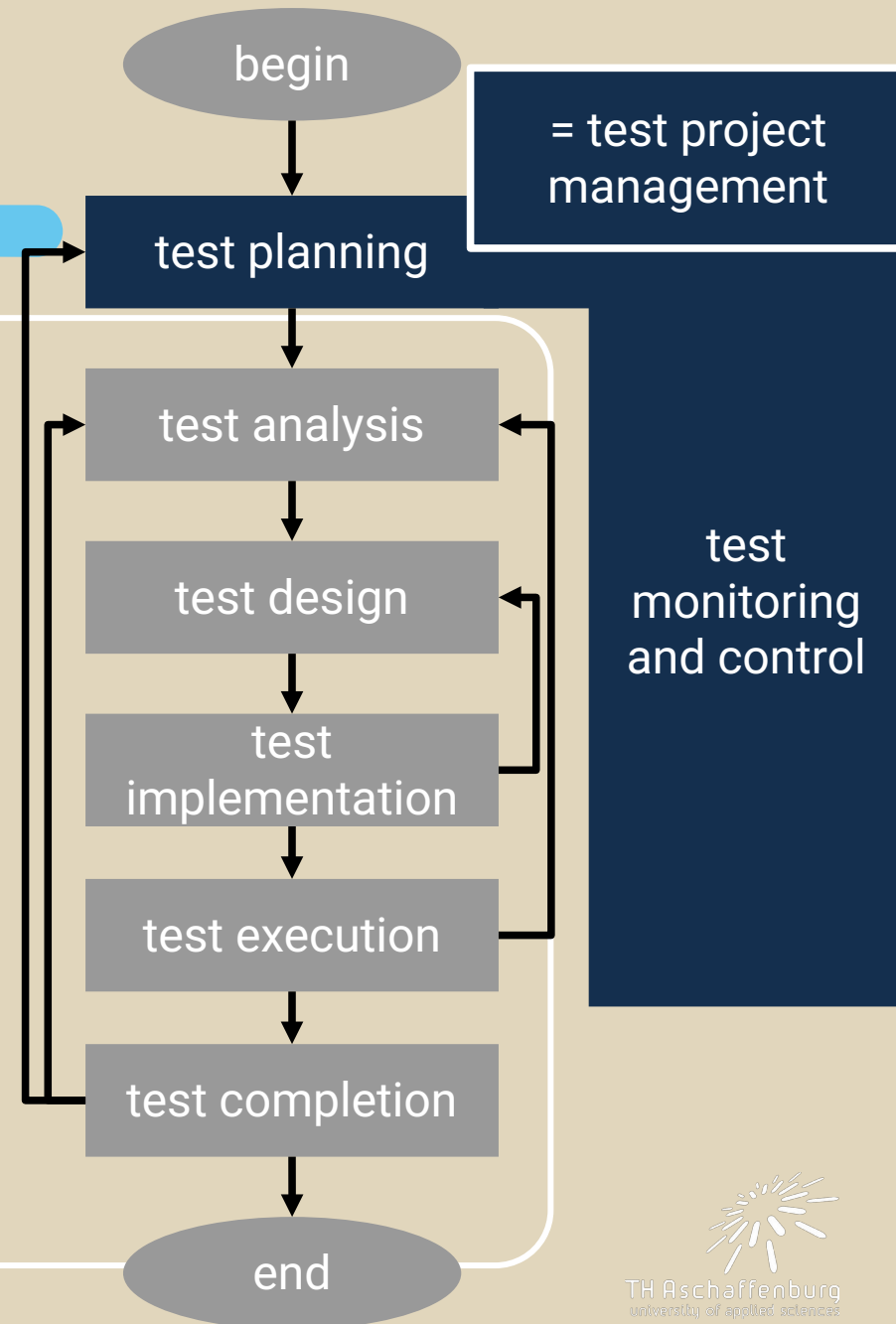
ITSQB
Syllabus

Test Process Artifacts



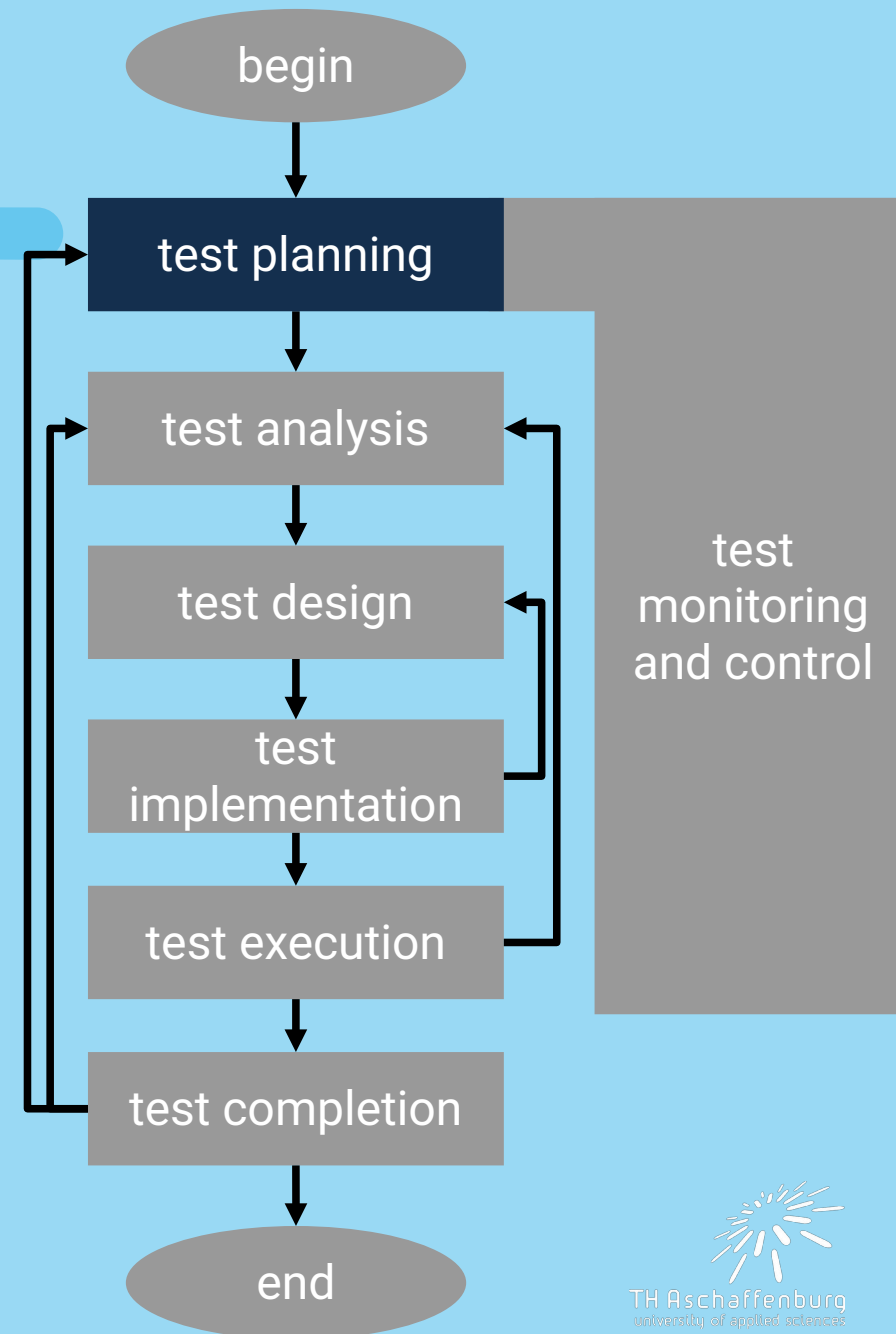
Question

What goals and activities did you find?



Test Planning – Activities

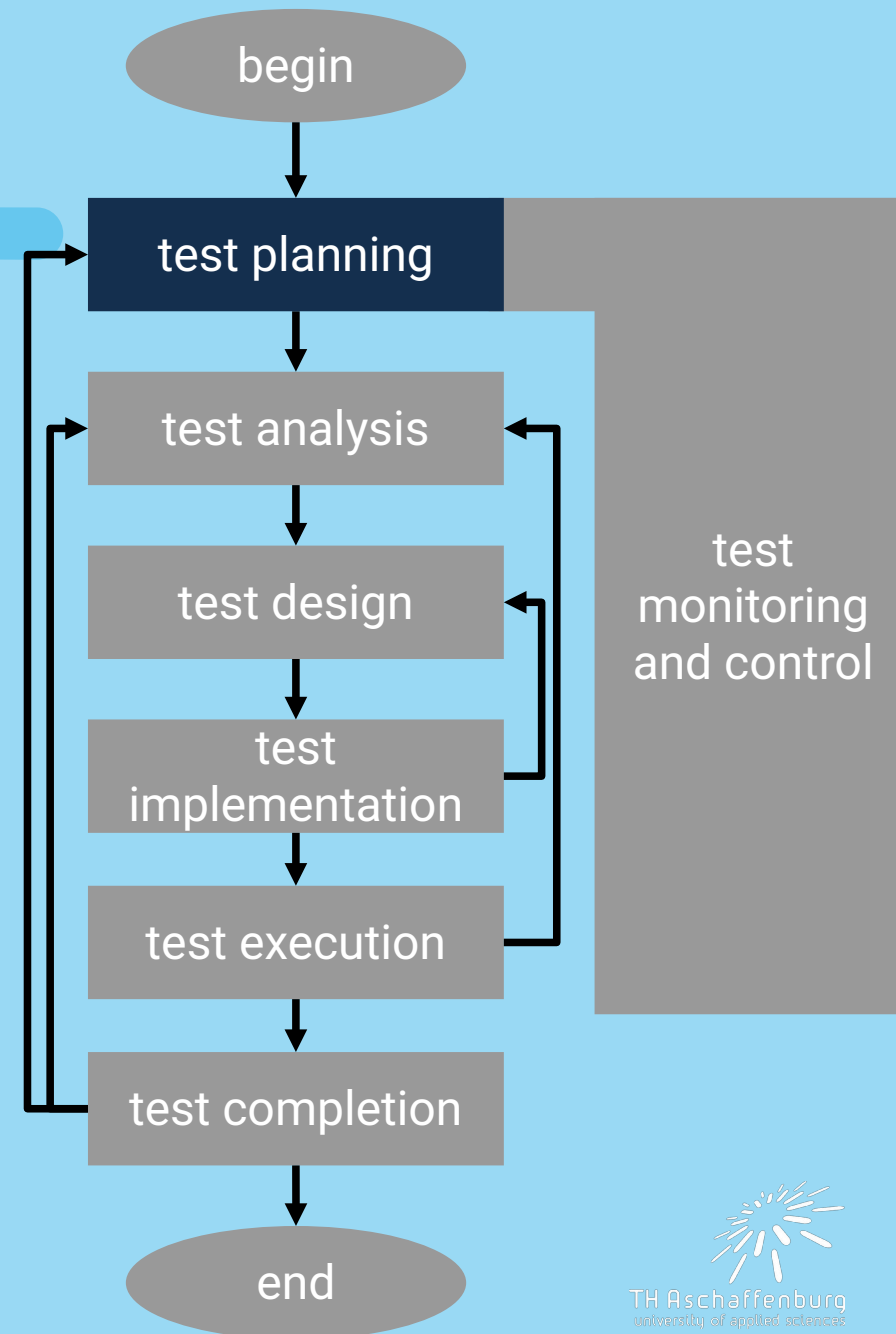
- Test strategy definition
 - Defining test objectives, scope, risks of testing
 - Specify test procedure
 - (techniques, test objects, coverage, test resources and teams involved in the test).
- Test planning in detail
 - Test resources (e.g. people, test environment, PCs).
 - Which part of the system should be tested (with what intensity)?
 - Which test procedures should be used?
 - Determine end criteria. What level of coverage should be achieved?
 - Prioritization of tests (accounting for the risk in the event of errors).
 - Plan tool support (deployment, procurement, training).
 - Determine rough test planning
- Record test planning and test strategy in the (master) test plan



Test Planning – Results

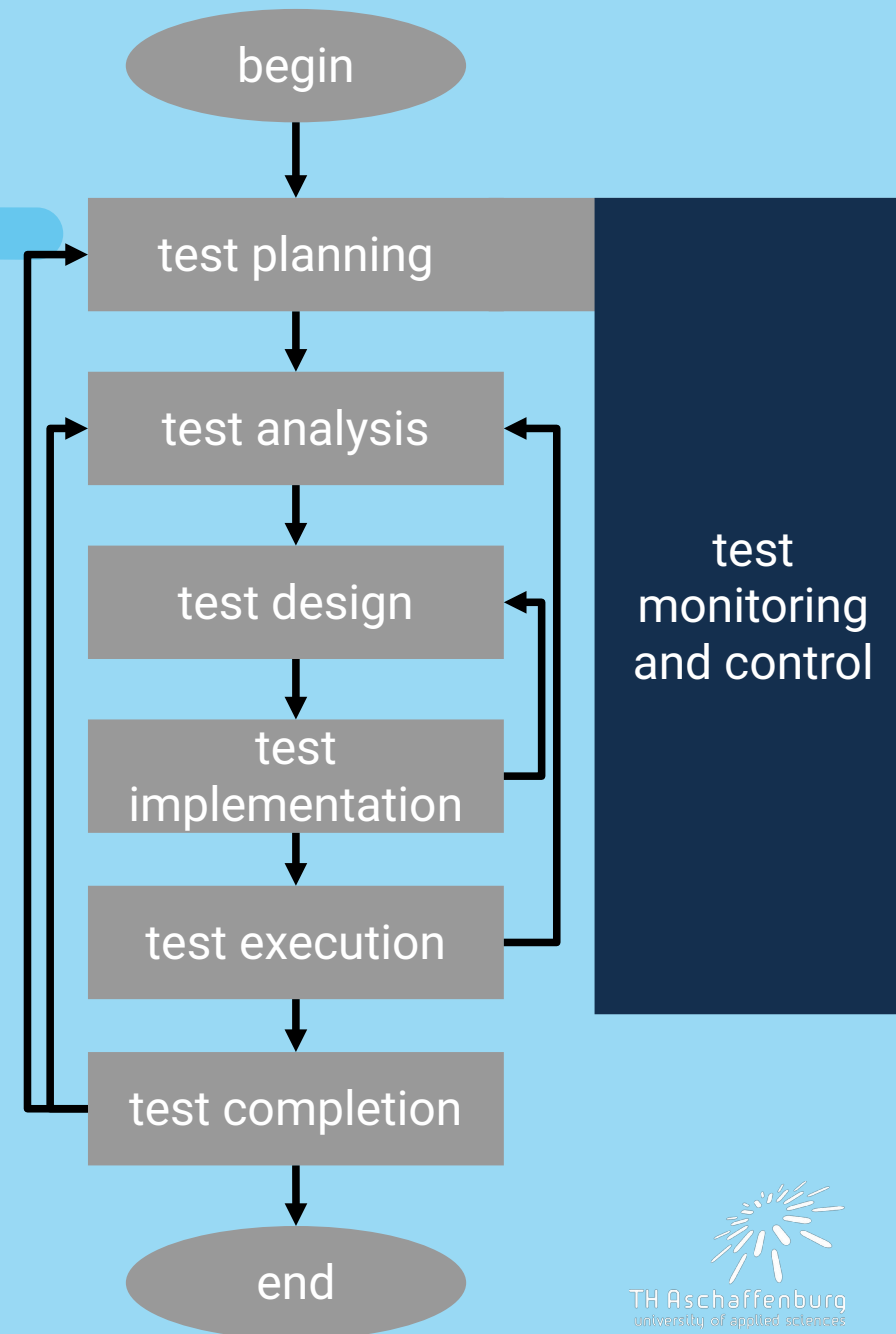
Test planning work results include **test concepts**

- Contents of a test concept:
 - Background/test objectives
 - Referenced documents
 - Test procedure
 - Reporting
 - Dependencies
 - End criteria
 - (...)



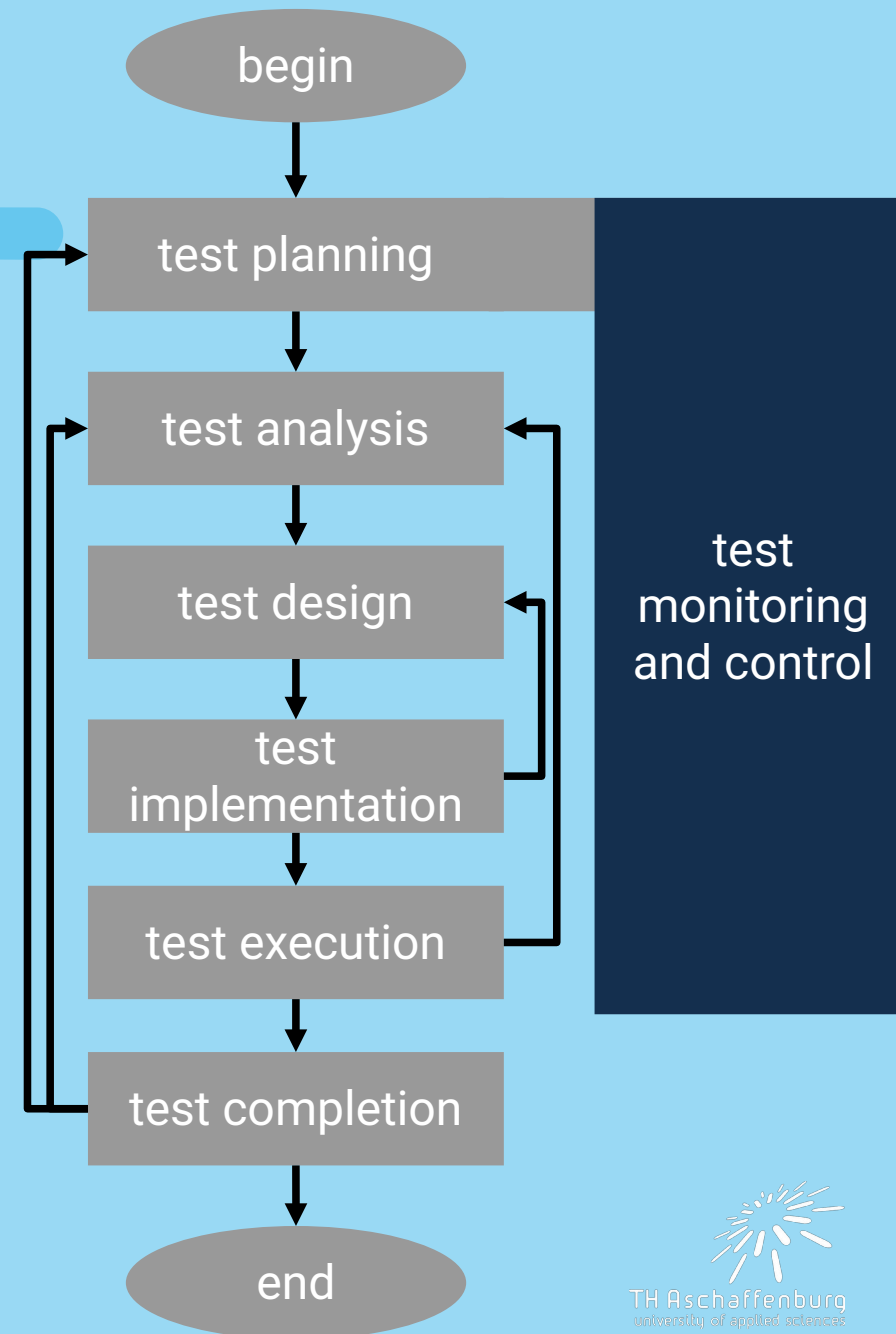
Test Monitoring and Control – Activities

- Test monitoring
 - includes comparison of actual progress with planned progress
 - use of monitoring metrics and end criteria defined in the test concept
- Test control
 - Taking the necessary measures to achieve the objectives defined in the test concept
- Test monitoring and control
 - Measuring and analyzing results
 - Monitoring and documenting test progress, test coverage achieved and the final criteria
 - Initiating corrective measures
 - Adjusting time and resource planning
 - Making decisions as to whether more tests are necessary



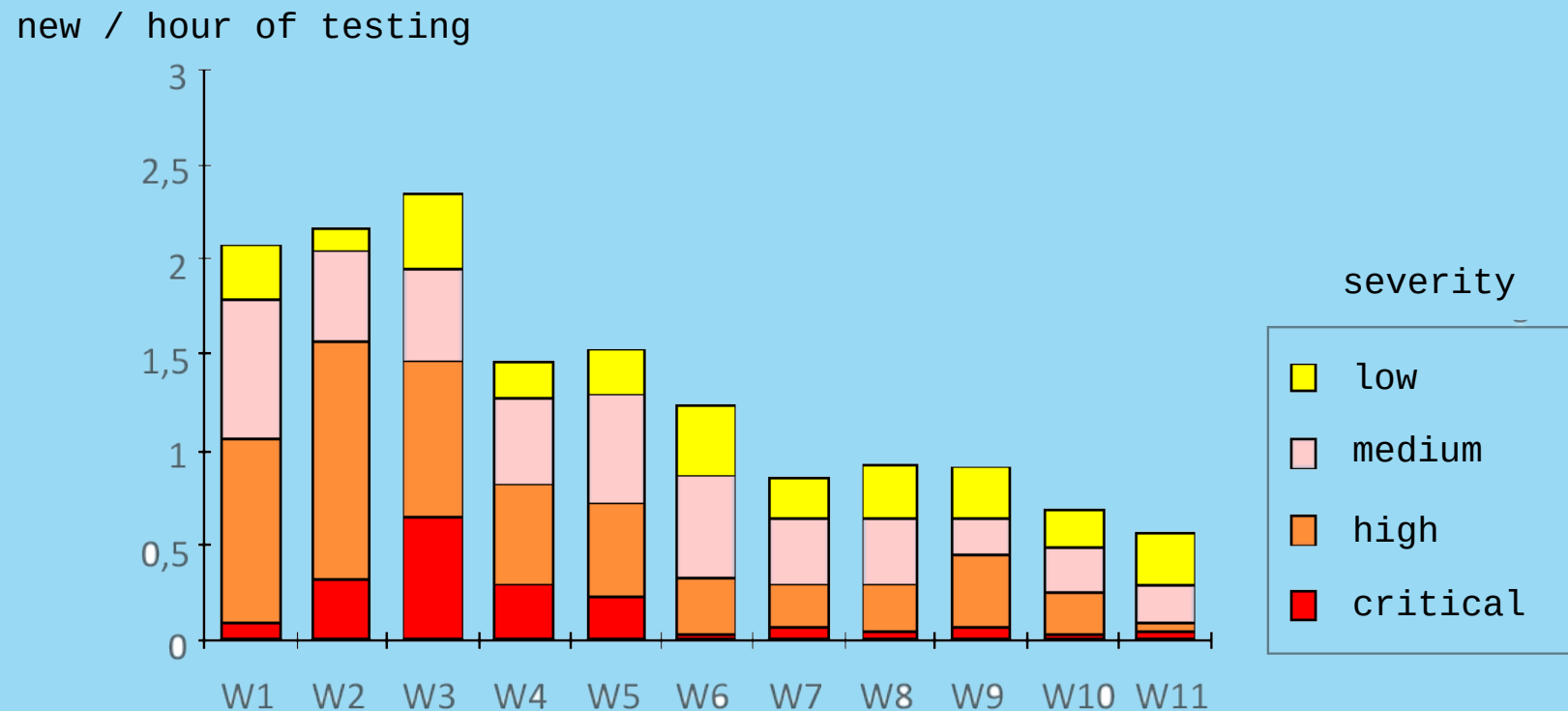
Test Monitoring and Control – Results

- Results typically include various types of test reports, including
 - **Test progress reports**
(produced on a continuous and/or periodic basis)
 - **Test completion reports**
(produced at various completion milestones)
- Content:
 - Details about the test progress at the time of the report (specific for the target audience)
 - Test execution results as soon as they are available.



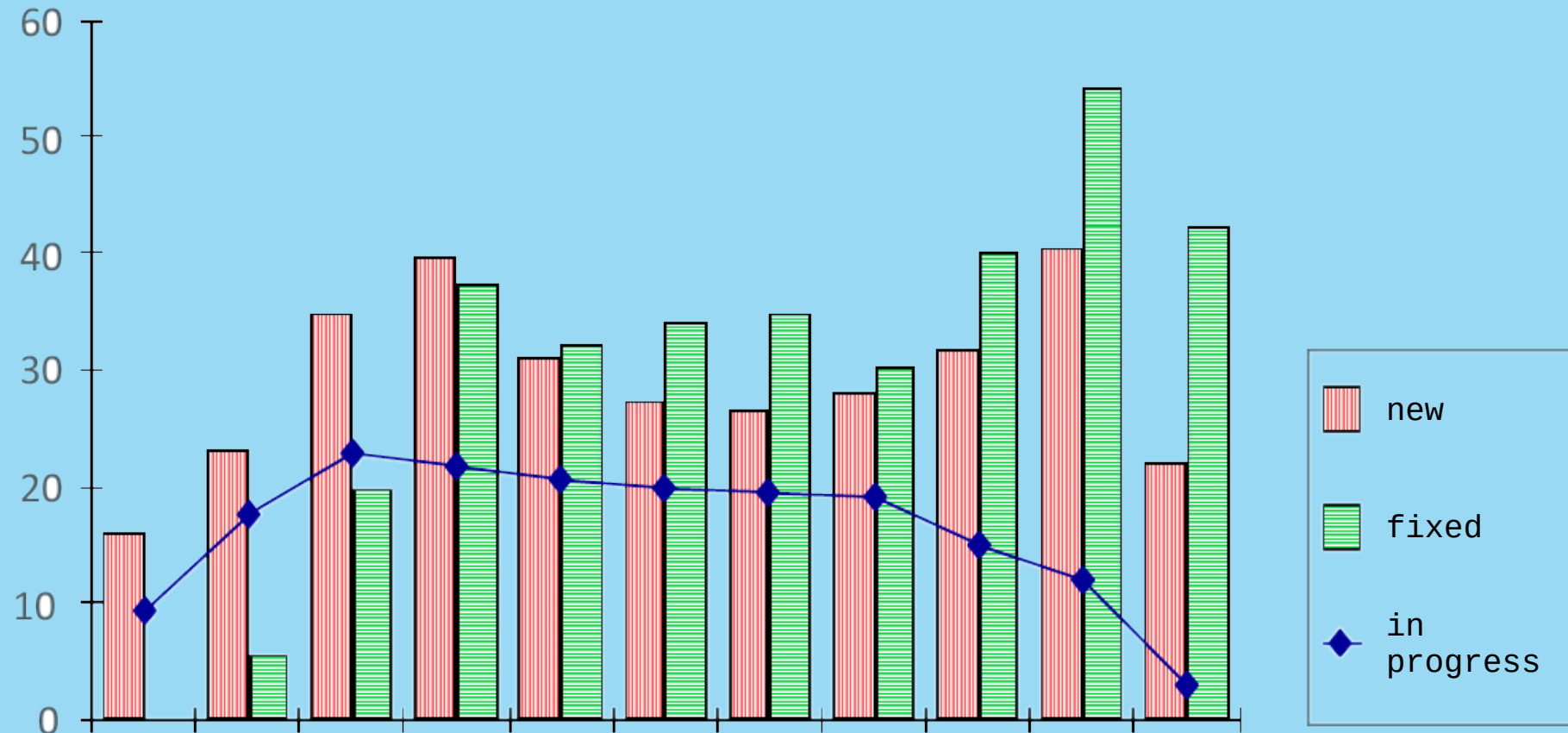
Test Monitoring and Control – Example Results (1/2)

- Example **end criteria**: maximum number of errors found by severity, e.g., no critical errors, max 2 high priority errors, etc.



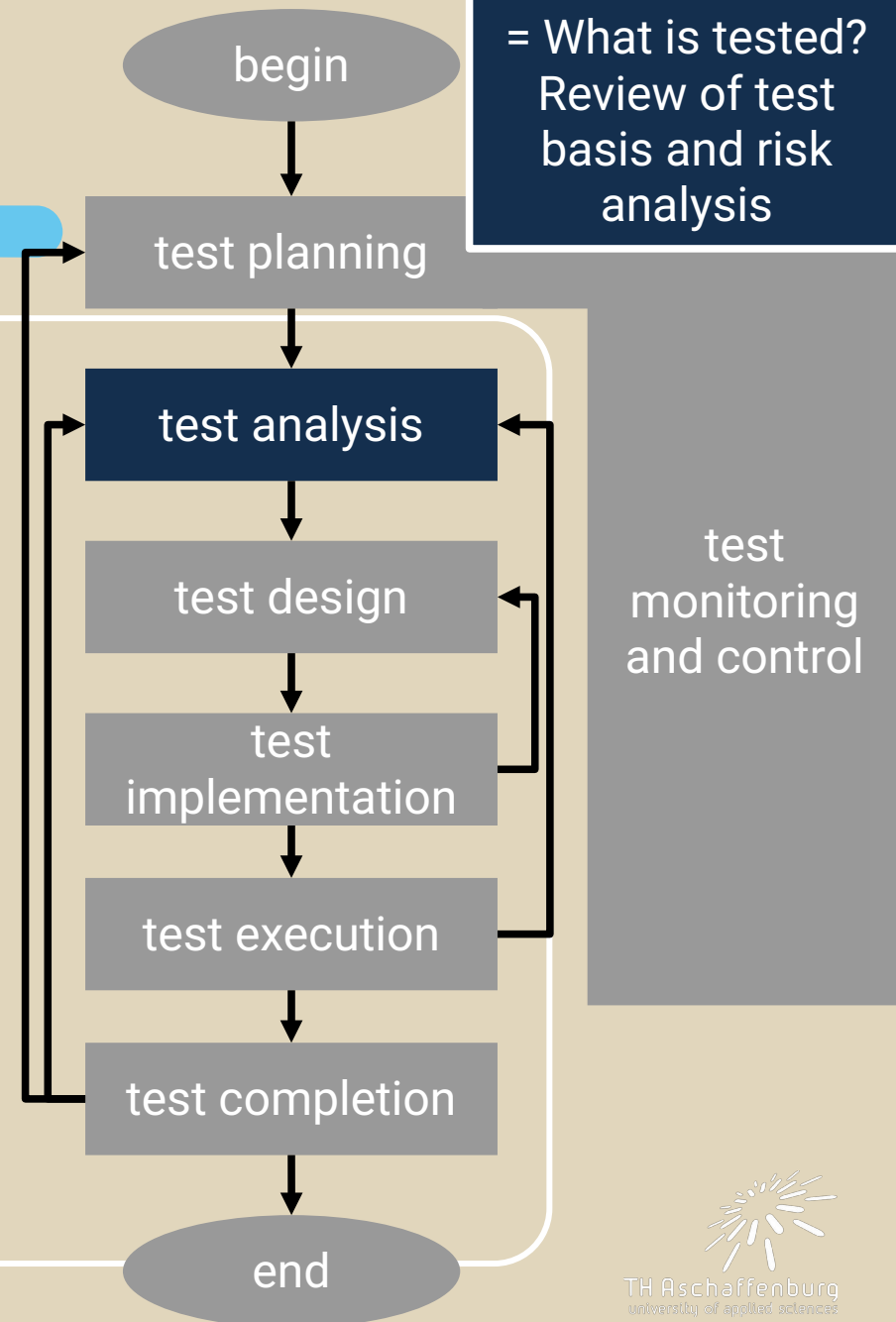
Test Monitoring and Control – Example Results (2/2)

- Example **end criteria**: Compare found / fixed errors



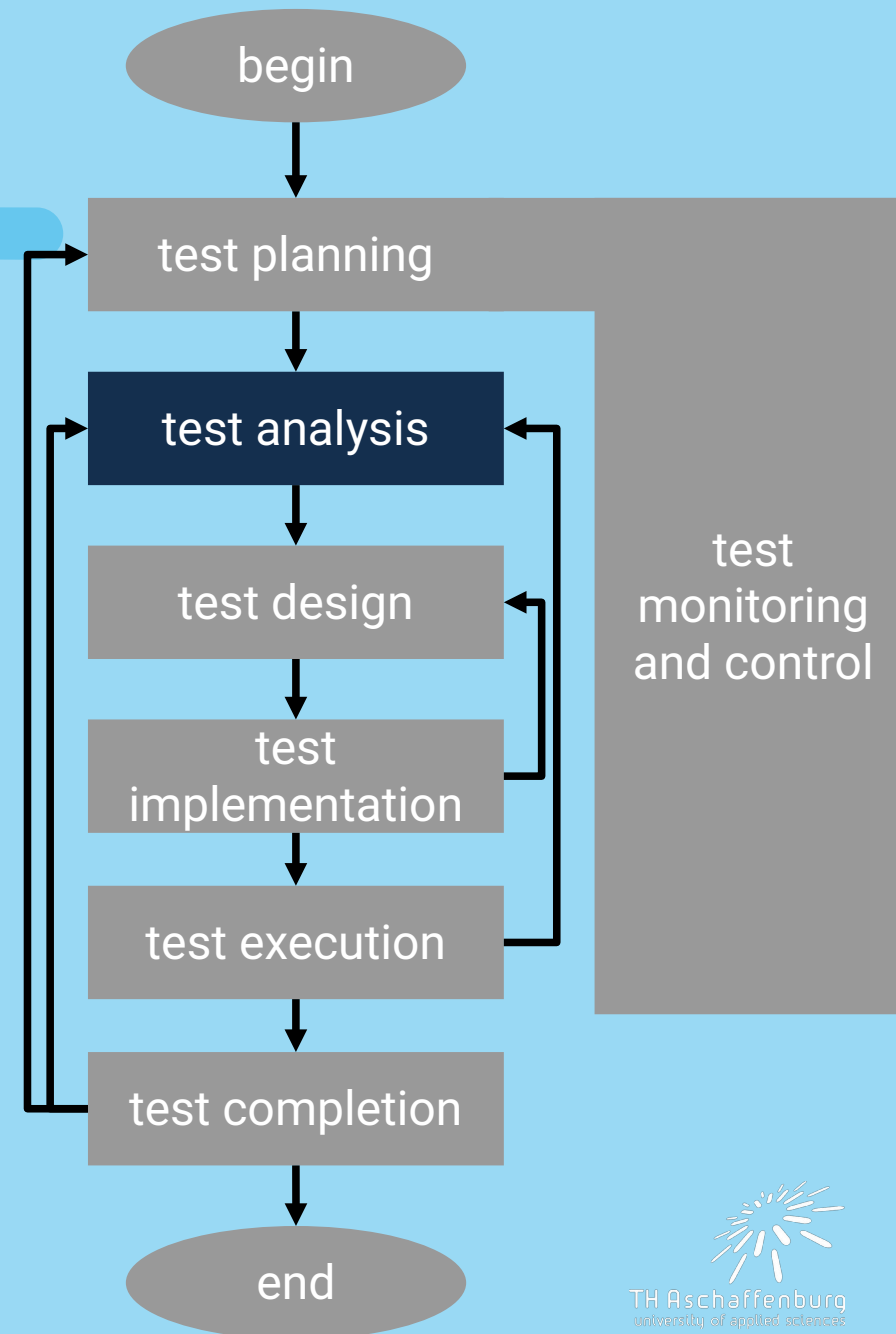
Question

What goals and activities did you find?



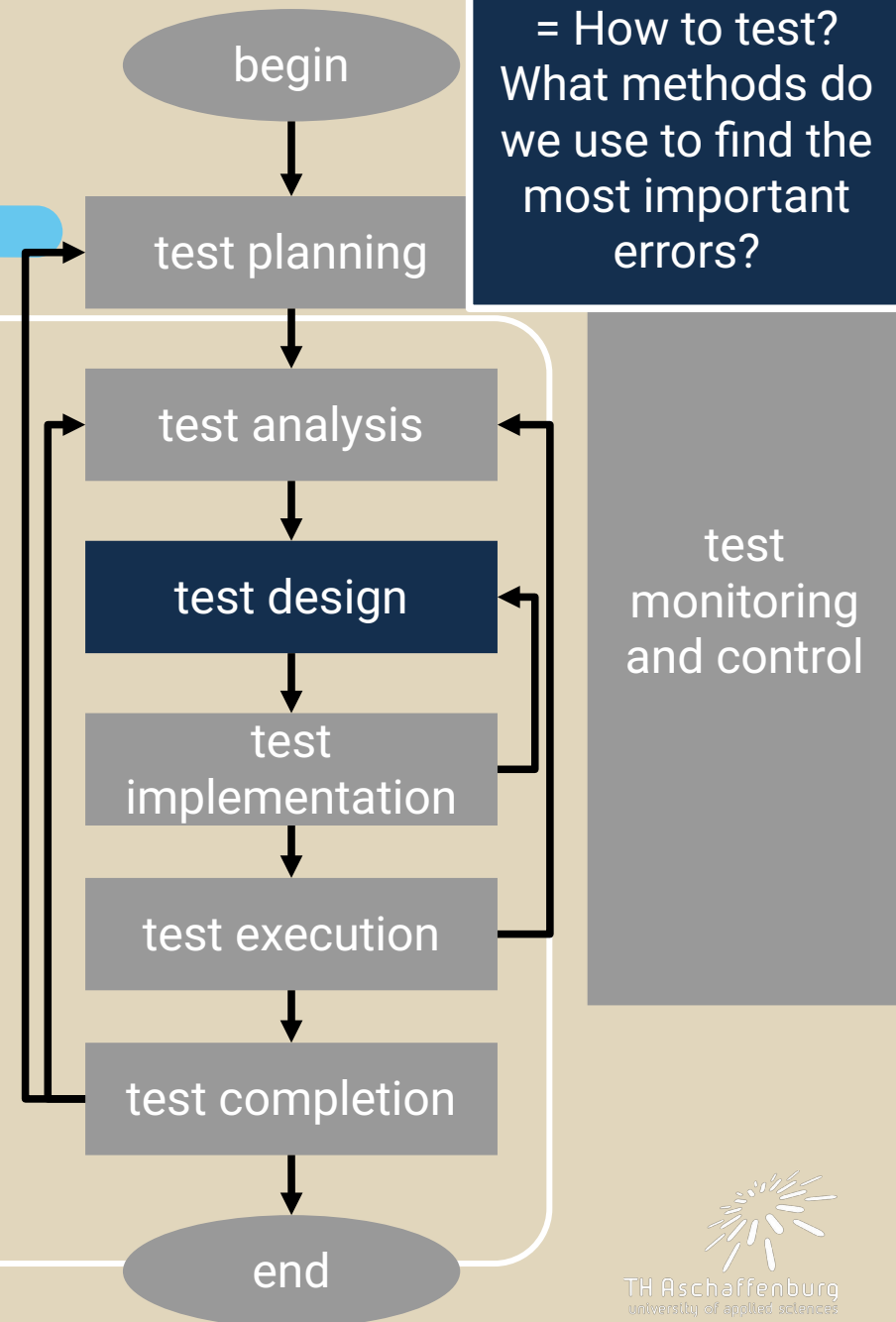
Test Analysis – Activities

- “What to test”
 - Review of test basis (requirements, architecture, design, interfaces, ...).
 - Evaluation of requirements and system testability
 - Risk analysis, taking into account functional, non-functional and structural aspects of the software
 - Identification of features and feature sets that need to be tested
 - Detailing test objectives into concrete test conditions
 - Definition and prioritization of test conditions for each feature
 - Capturing bi-directional traceability between each element of the test basis and the associated test conditions
- Results
 - Defined and prioritized test conditions
 - Test conditions can be traced to the test basis’ elements



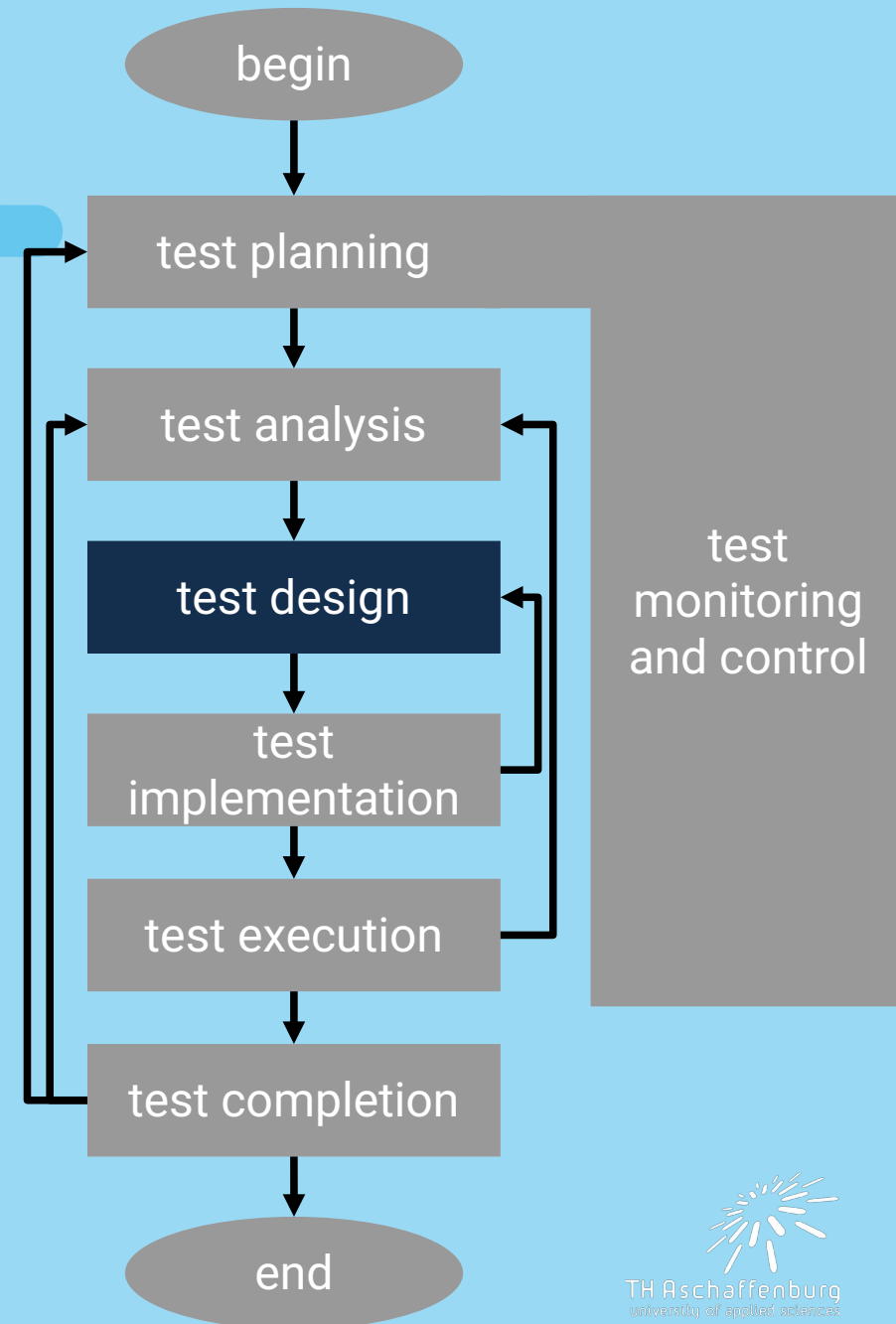
Question

What goals and activities did you find?



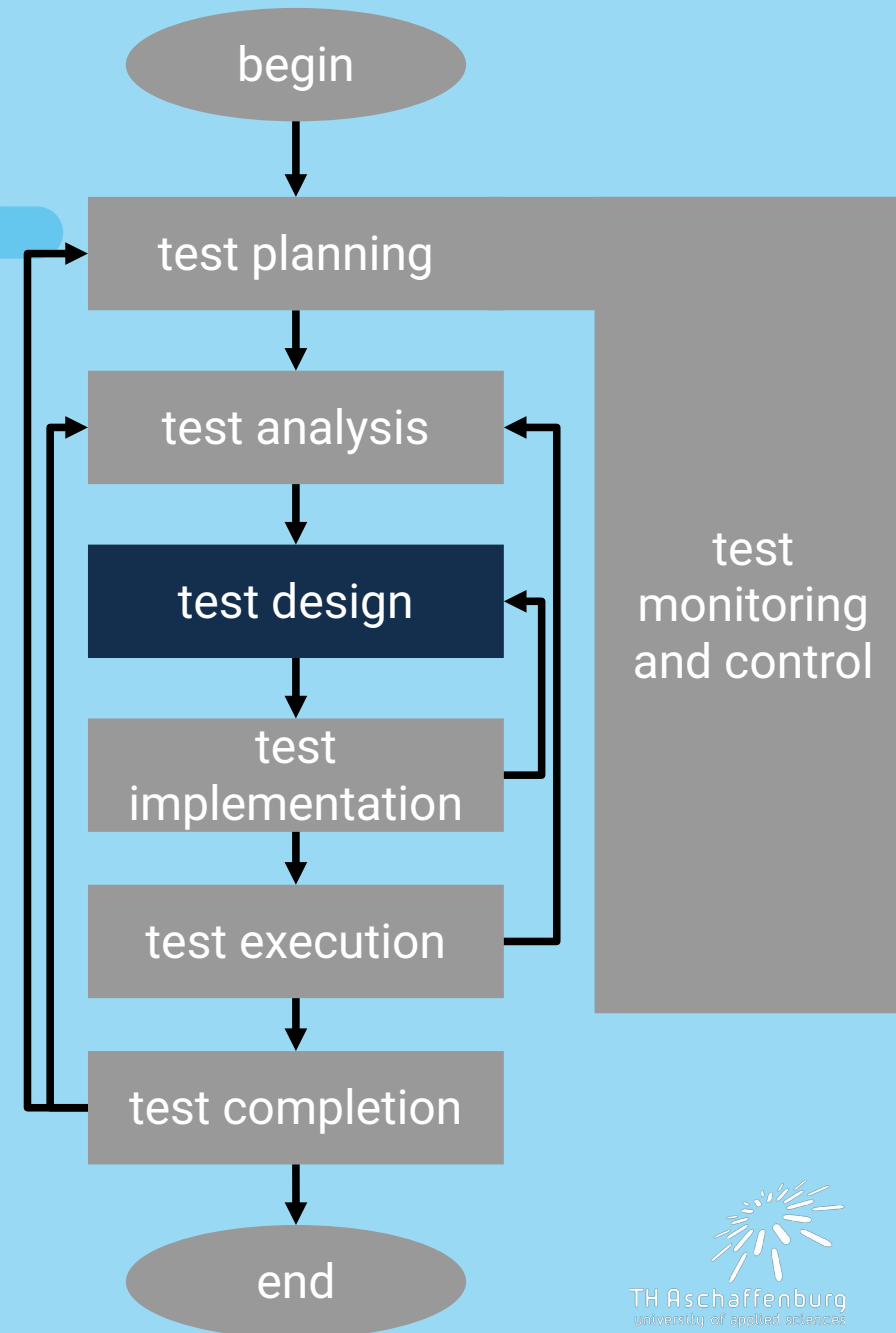
Test Design – Activities

- **Test design procedure:** Specification of (sets of) test cases
- **Prioritization** of (sets of) test cases
- Identification of necessary **test data**
- Design of the **test environment** (infrastructure, tools, ...).
- Ensuring **bidirectional traceability**



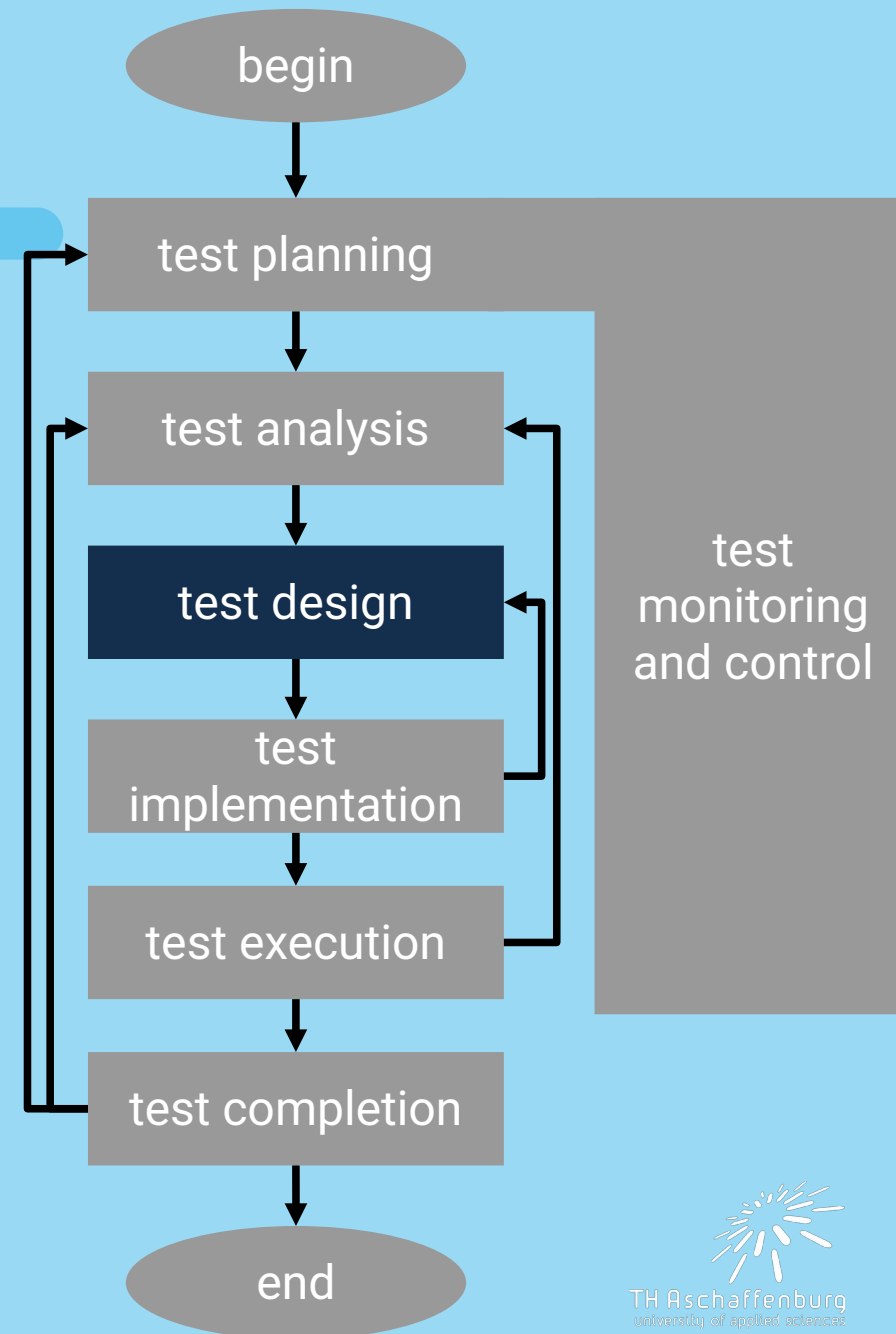
Test Design – Activities

- **Test cases are more than just test data**
 - Pre-condition (fulfilled before test)
 - Expected result (define before test execution!)
 - Post-condition (fulfilled after test)
- **Linking test cases to test suites**
 - Test case: only a few elementary processing steps
 - Pay attention: which post-conditions of test cases fulfill which preconditions of test cases?



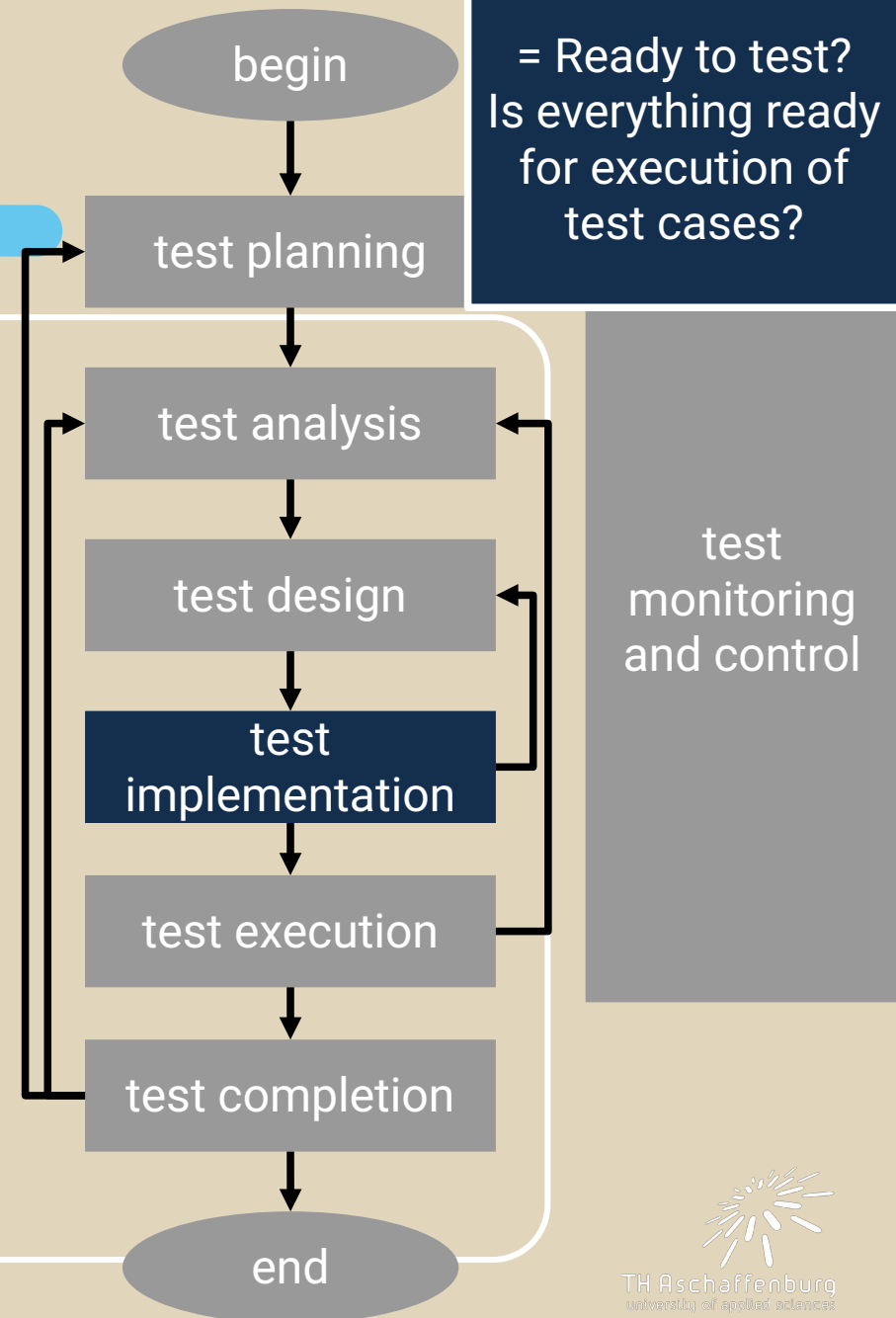
Test Design – Results

- (Sets of) **test cases** that cover the test conditions defined during test analysis.
- Bidirectional traceability to test conditions is important
- Hint: design **abstract/logical test cases** without concrete values for input data and expected results. Increases reusability!
- Design/identification of necessary **test data**
- Design of the **test environment**
- Identification of **infrastructure and tools**
- The extent to which these results are documented may vary



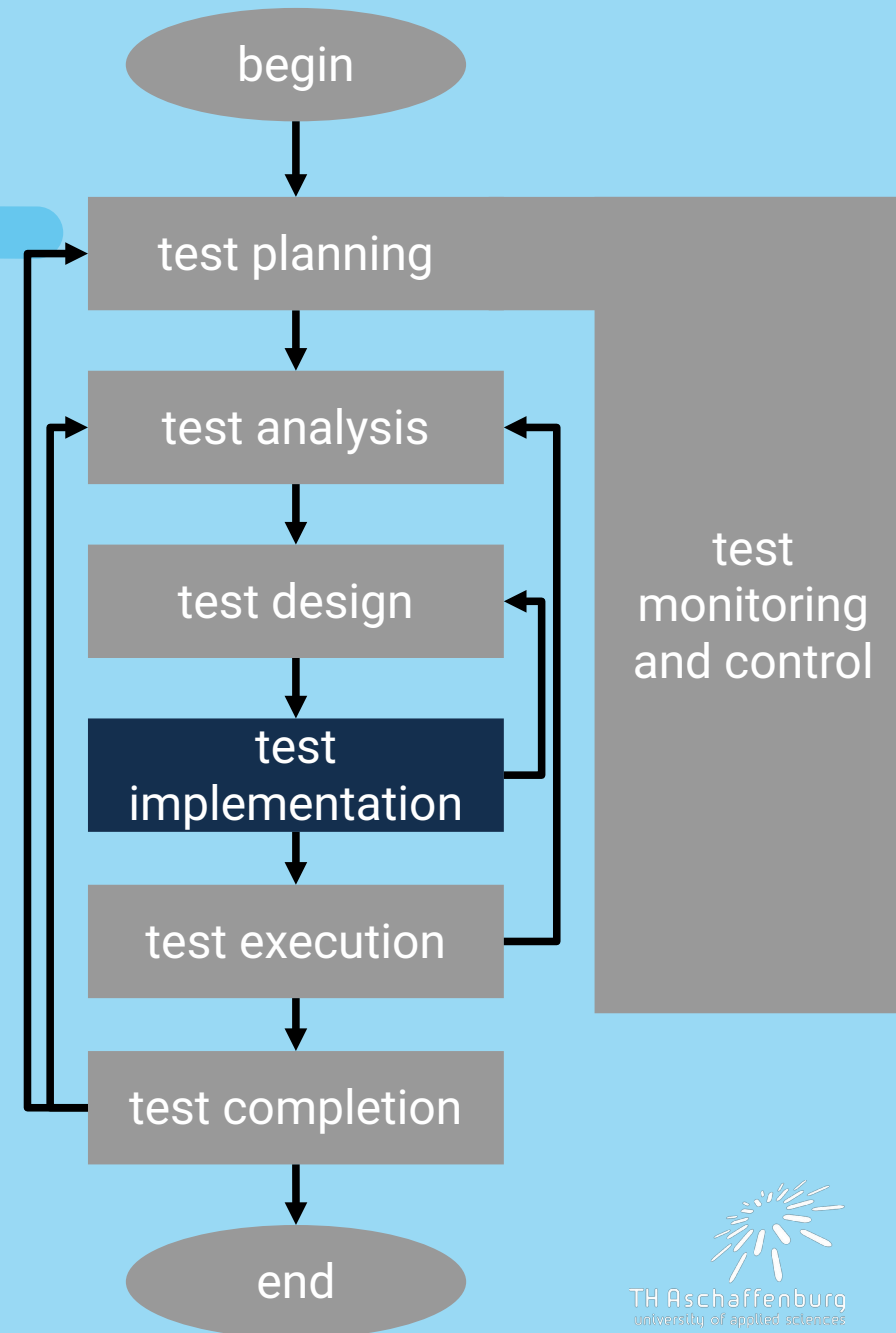
Question

What goals and activities did you find?



Test Implementation – Activities

- Development and prioritization of **test scripts**
- Compilation/creation of **test suites** from the test procedures.
- Arranging the test suites within a **test execution schedule**
- Setting up the **test environment**
- Verifying that everything required is set up correctly
- Preparing test data and ensuring that it is properly loaded into the test environment
- Verifying and updating bi-directional traceability
- The **creation of automated test scripts**



Test Implementation – Results

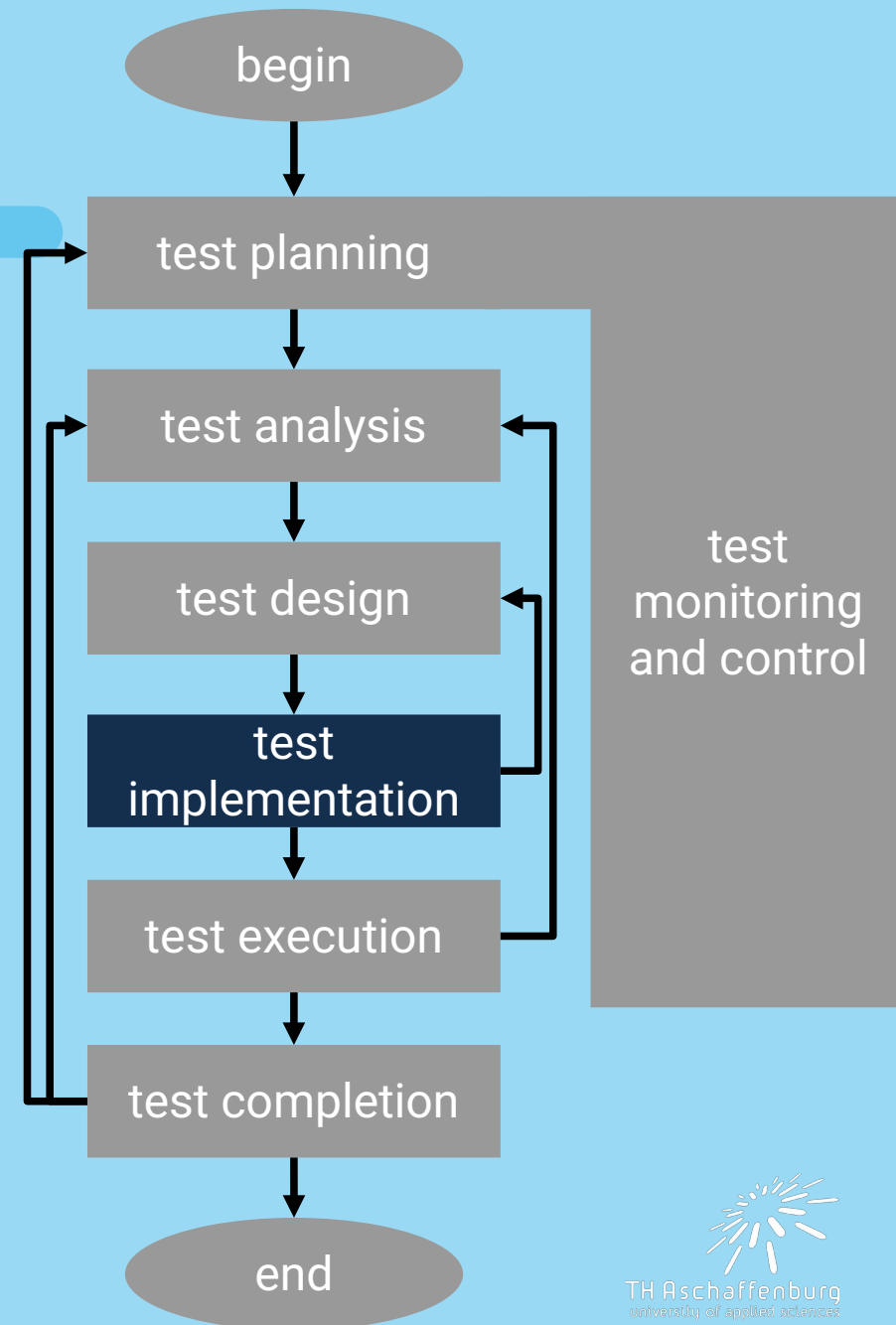
Test scripts and sequence of those scripts

- **Test Suite**

A set of test scripts or test procedures to be executed in a specific test run.

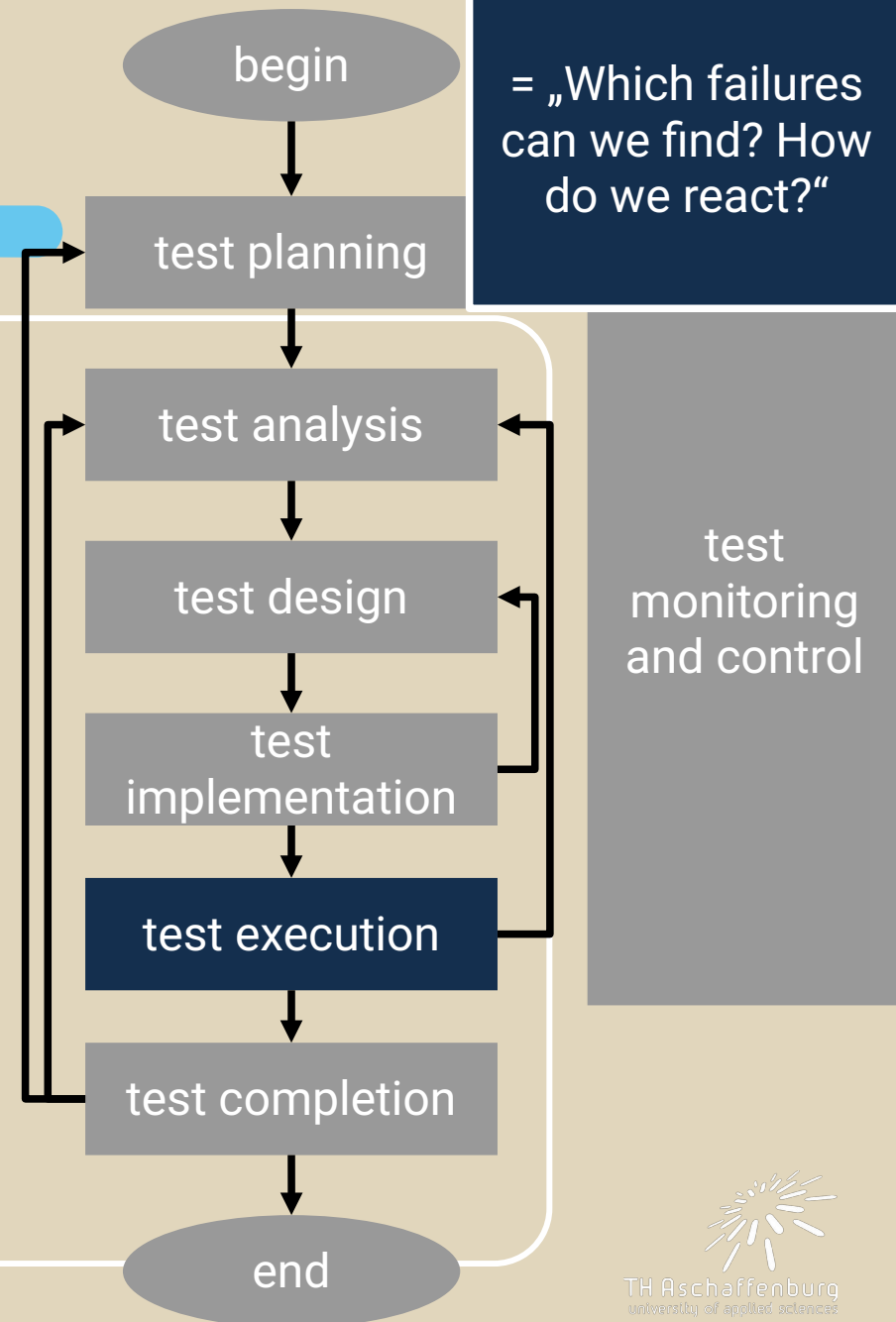
- **Test Execution Schedule**

A schedule for the execution of test suites within a test cycle.



Question

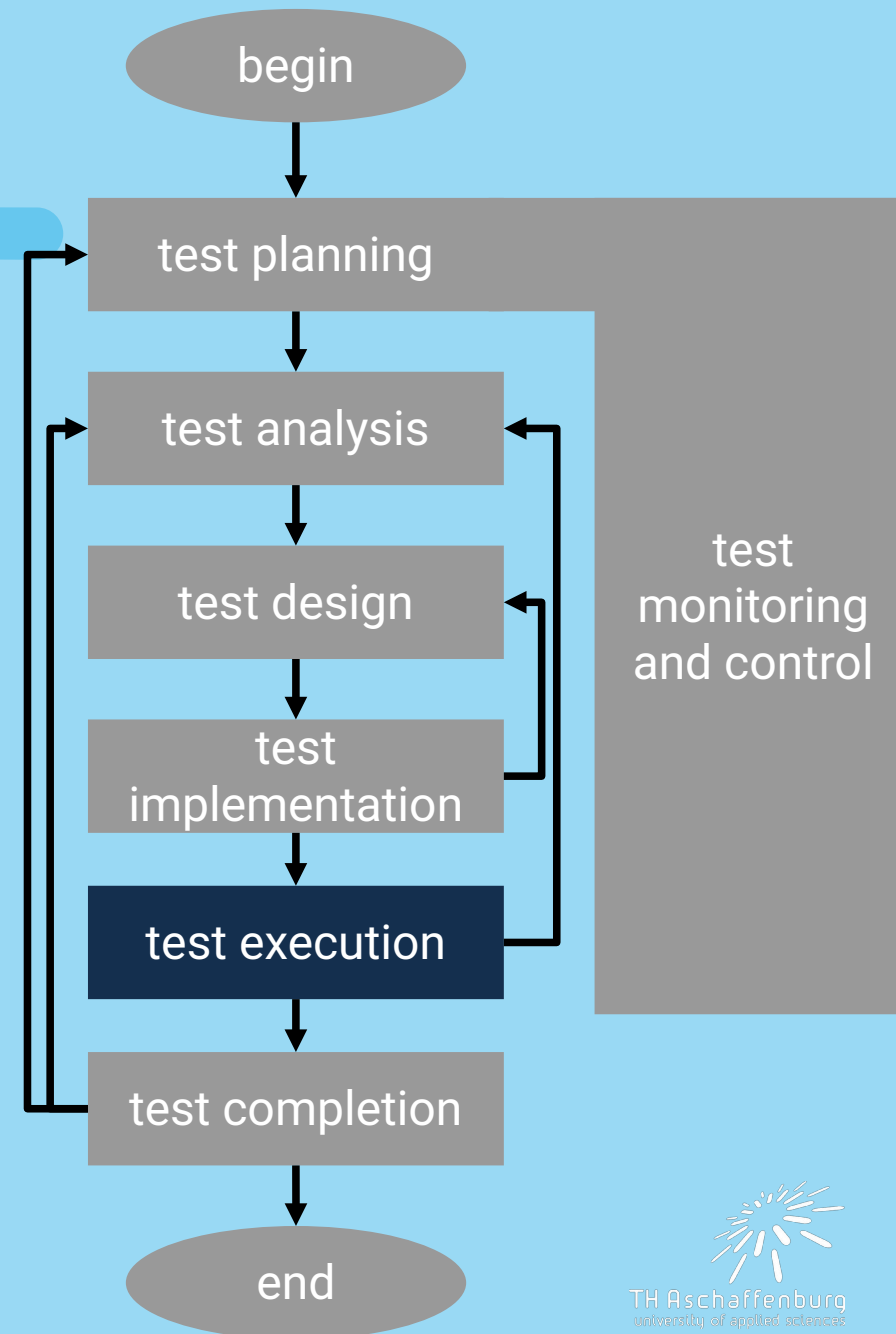
What goals and activities did you find?



Test Execution – Activities (1/2)

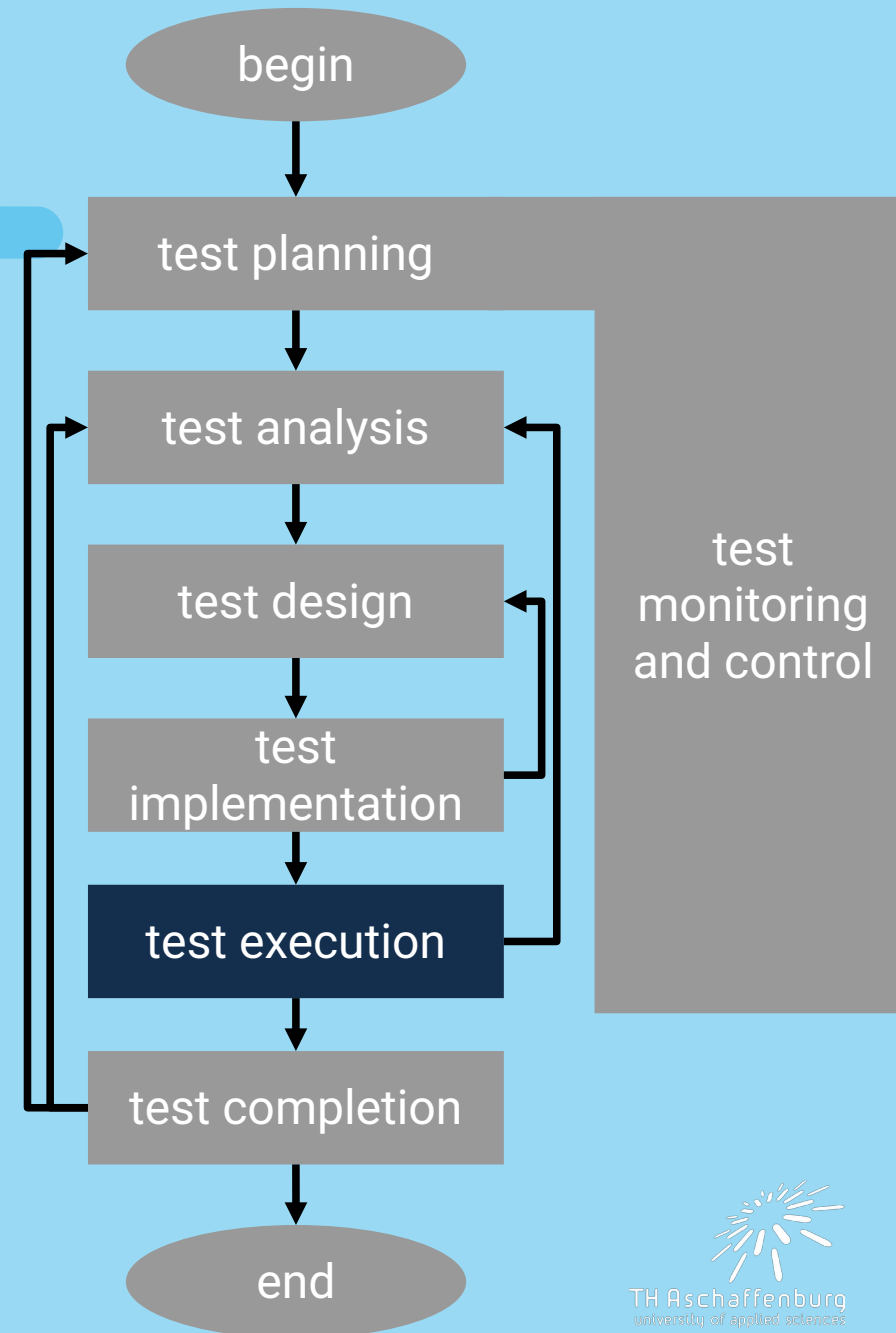
Execute test suites according to the test execution plan.

- Record **IDs and versions** of the test subject's test elements, the test tools and test resources
- Execute the tests either manually or by using test execution tools
- **Compare actual results with expected results**
- Determine defect severity (defect class)
- Determine priority of fixes
- Analyze the causes of anomalies (e.g. errors in the code, errors in the test case, false positive results, faulty test environment)
- Report on error conditions based on observed failures



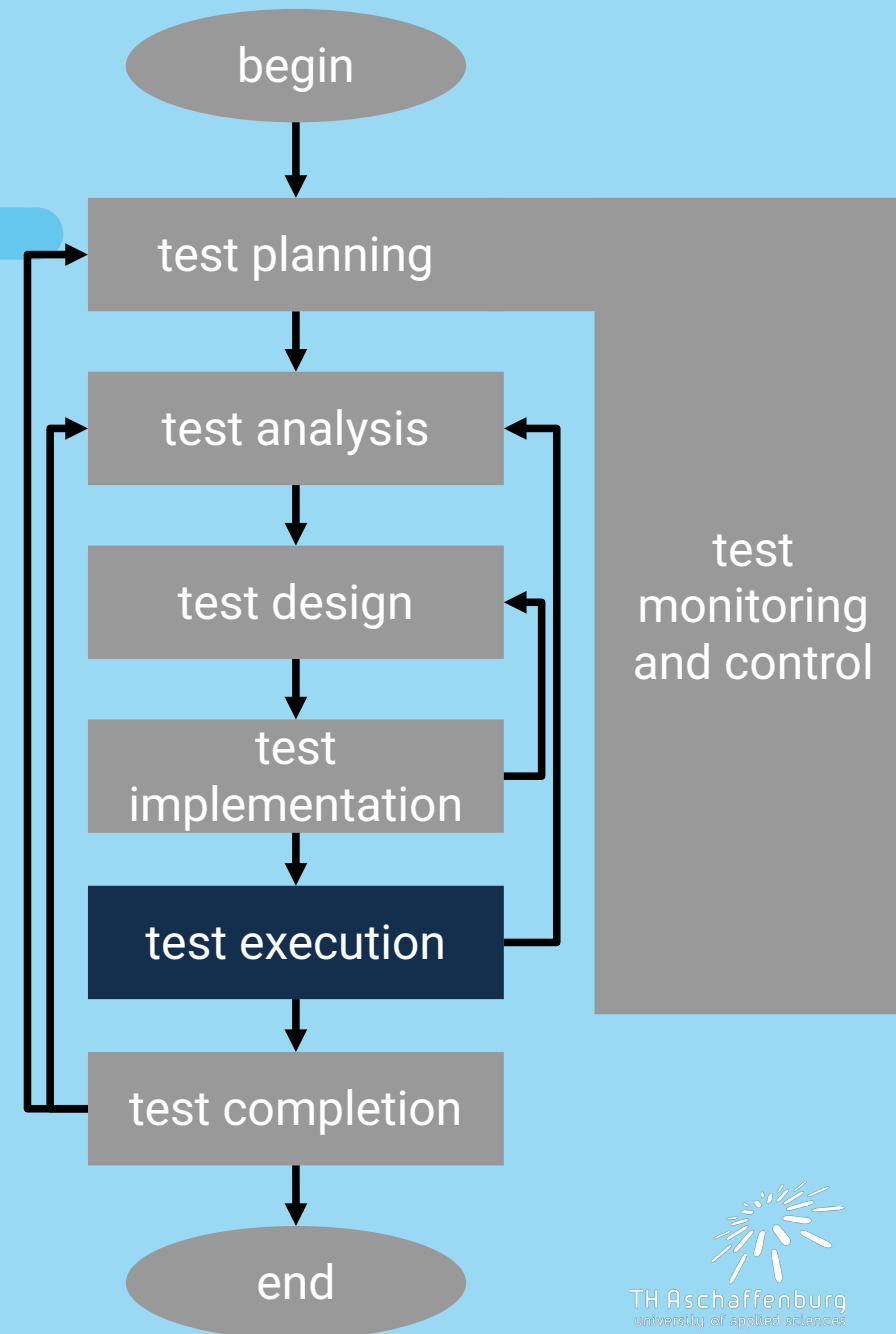
Test Execution – Activities (2/2)

- Record results of test execution (e.g. passed, failed, blocked)
- Repeat test activities (e.g. corrected test, error retests, regression tests)
- Verification and updating of bidirectional traceability between:
 - test basis
 - test conditions
 - test cases
 - test procedures
 - test results
- Got to end of test?
- Degree of coverage achieved?
 - Problem: unreachable code



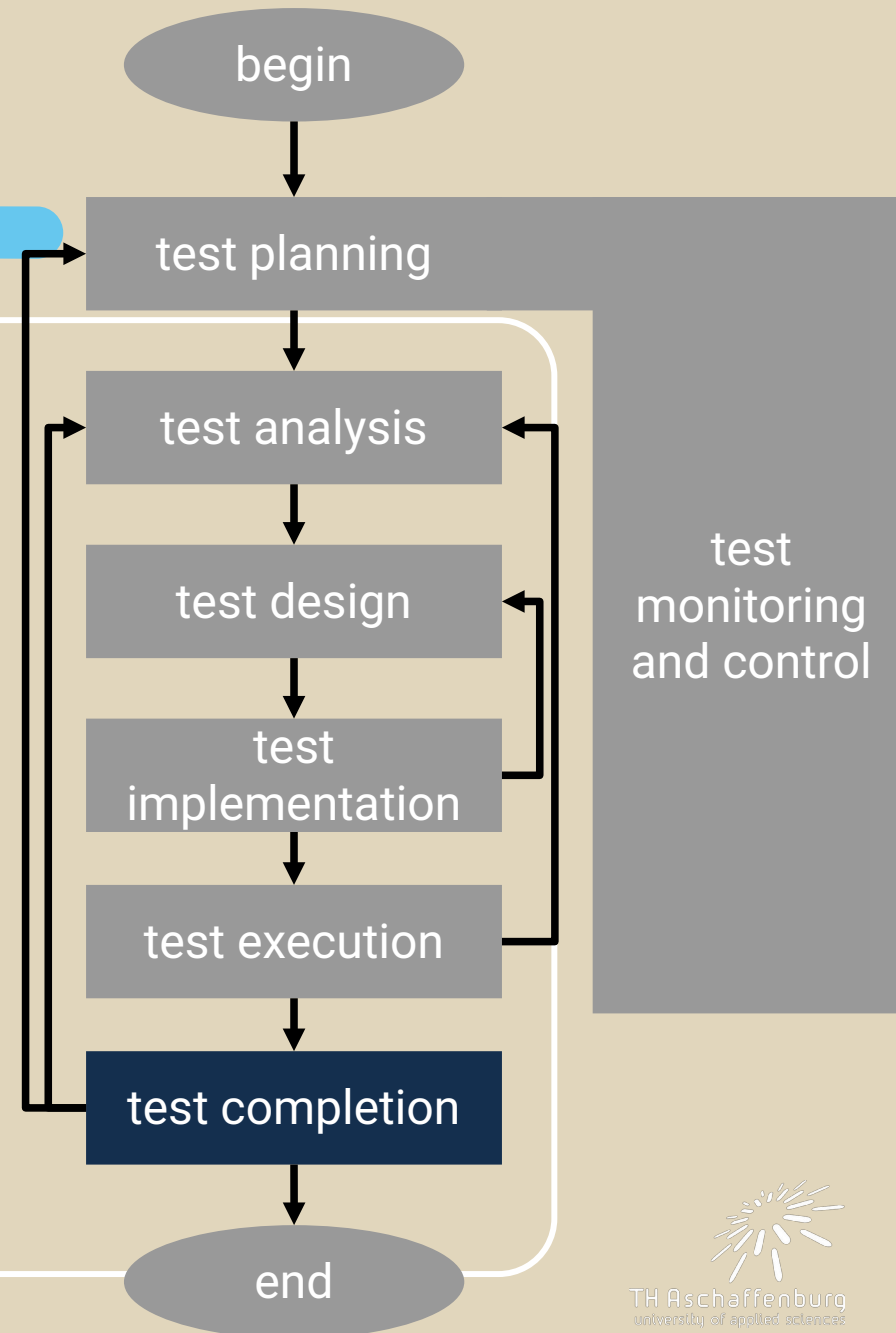
Test Execution – Results

- Documentation of status of individual test cases or test scripts (e.g. executable, passed, failed, blocked, planned, skipped, etc.)
- Error reports, if applicable
- Test log:
 - What was tested when, by whom, with which intensity and results?
 - Test execution must also be comprehensible for persons not involved
 - Test elements, test tools and test equipment used
 - Evaluation with regard to the end criteria defined in test concept
- Proof that the planned test strategy was actually implemented
- Further effort justified?
 - Factors for the end of testing in practice: time and costs
 - Decision: more tests? Adapt defined end criteria?



Question

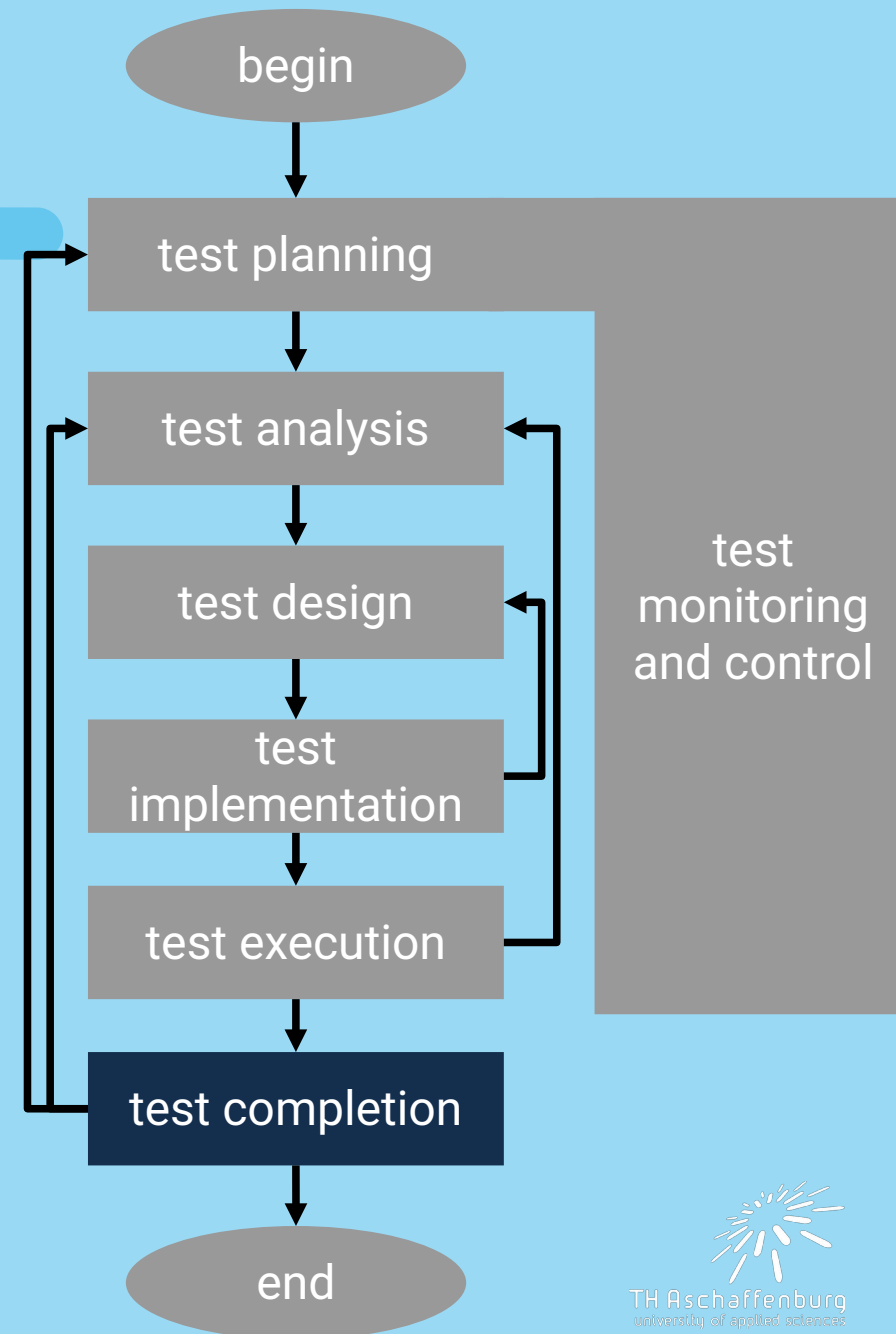
What goals and activities did you find?



Test Completion – Activities

Collect data from completed test activities to consolidate experience, test resources and other relevant information.

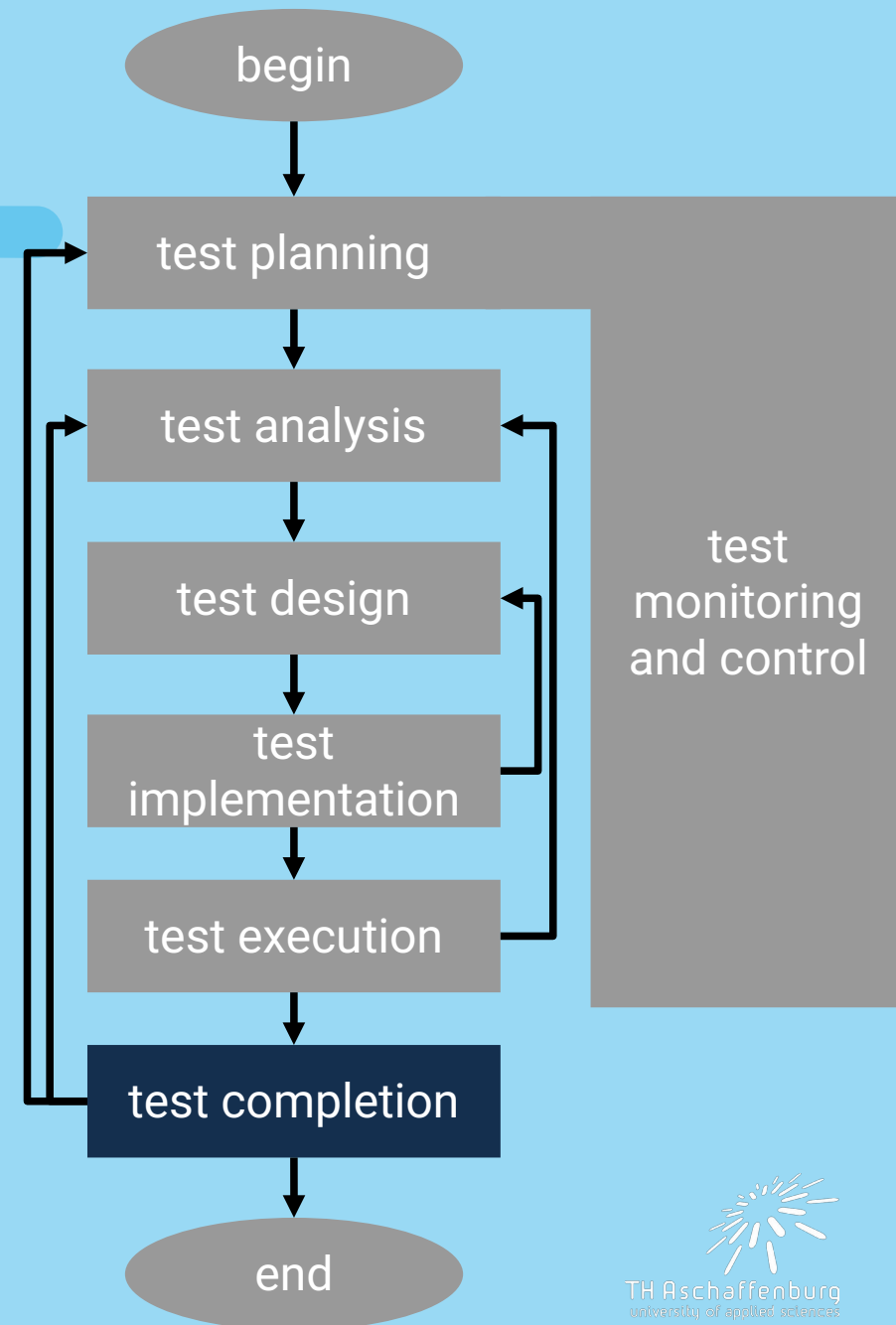
- Check
 - All error reports completed?
 - Create change requirements for existing errors
- Finalize and archive the test environment, test data, test infrastructure and other test assets for future reuse
- Handover of the test resources to maintenance teams, other project teams or stakeholders
- “Lessons learned”: analyze findings, determine necessary changes for future iterations, product releases and projects
- Improve the test process by implementing findings
- Preparation of final test report



Test Completion – Results

Collect data from completed test activities to consolidate experience, test resources and other relevant information.

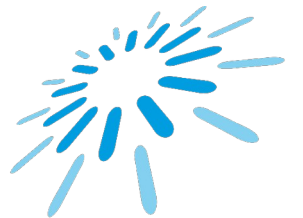
- Final test reports
- Collection of open issues for improvement in next projects
- Finalized test resources





Until next time!

Collaboration, Quality, Test
Prof. Dr. Jóakim von Kistowski



TH Aschaffenburg
university of applied sciences